

Implementacja gry planszowej Catan oraz agentów AI do rozgrywki 1v1

(Implementation of the Catan board game and AI agents for 1v1 gameplay)

Maciej Stempniak

Yaryna Rachkevych

Praca inżynierska

Promotor: dr Artur Kraska

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

30 stycznia 2026

Streszczenie

Abstract

Spis treści

1. Wstęp	9
1.1. Wprowadzenie do tematyki gier planszowych i sztucznej inteligencji .	9
1.2. Gra Catan – charakterystyka i potencjał badawczy	10
1.3. Plansza, cel gry i podstawowe zasady	10
1.4. Produkcja zasobów i miara pipsów	13
1.5. Wariant 1 vs 1 przyjęty w pracy	14
1.6. Zalety gry <i>Catan</i>	14
1.6.1. Scena turniejowa i mistrzostwa	15
1.6.2. Potencjał badawczy (w kontekście botów)	15
1.7. Cel i zakres pracy	15
1.8. Podział pracy i wkład autorów	16
1.9. Struktura pracy	17
2. Przegląd istniejących rozwiązań	19
2.1. Istniejące implementacje Catana	19
2.1.1. Catanatron	19
2.1.2. JSettlers	19
2.1.3. CatanAI i projekty eksperymentalne	20
2.1.4. Tabela porównawcza istniejących rozwiązań	20
2.2. Boty i algorytmy decyzyjne w grach planszowych	20
2.2.1. Monte Carlo Tree Search	20
2.2.2. Uczenie przez wzmocnianie	20
2.2.3. Heurystyki i podejścia regułowe	21

2.3. Podsumowanie i uzasadnienie autorskiego podejścia	21
2.3.1. Uzasadnienie wyborów projektowych	22
2.3.2. Podsumowanie	24
3. Analiza problemu i założenia projektowe	25
3.1. Wymagania funkcjonalne gry	25
3.2. Wymagania нефunkcjonalne (wydajność, testowalność)	27
3.3. Model stanu gry i akcji	28
3.3.1. Stan gry	28
3.3.2. Akcja jako nośnik decyzji	28
3.3.3. „Jedno źródło prawdy” dla reguł	29
3.3.4. Cofanie akcji i symulacje stanów pośrednich	29
4. Architektura i implementacja systemu	31
4.1. Wybór technologii i narzędzi	31
4.2. Struktura projektu	32
4.3. Implementacja planszy i elementów gry	32
4.4. System akcji i cofania ruchów	34
4.5. Zarządzanie stanem gry	35
5. Symulowanie i odtwarzanie rozgrywek	37
5.1. Uruchamianie serii rozgrywek	37
5.2. Format rejestracji rozgrywek (JSONL)	37
5.3. Odtwarzanie i wizualizacja (replay viewer)	38
5.4. Jak przeprowadzać rozgrywki	38
6. Testowanie i weryfikacja poprawności	41
6.1. Strategia testowania	41
6.2. Testy jednostkowe i integracyjne	41
6.3. Walidacja zgodności z zasadami gry	42
7. Projekt i implementacja botów	43

7.1. Gracz losowy (baseline)	43
7.1.1. Założenia projektowe	43
7.1.2. Rola w badaniach	44
7.1.3. Charakterystyka zachowania	44
7.1.4. Boty heurystyczne	45
7.2. Bot wykorzystujący algorytm alpha-beta	46
7.2.1. Reprezentacja drzewa i znaczenie głębokości	46
7.2.2. Ograniczanie rozmiaru drzewa przeszukiwania	47
7.2.3. Algorytm alpha-beta	47
7.2.4. Funkcja oceny pozycji i fazy gry	48
7.2.5. Symulacja rzutów kością w drzewie przeszukiwania	49
7.2.6. Heurystyki uzupełniające i mechanizmy awaryjne	50
7.3. Boty celujące w jedną strategię	50
7.3.1. Bot celujący w jeden zasób	50
7.3.2. Wybór priorytetowego surowca	51
7.3.3. Ustawienie początkowe: wymuszenie portu 2:1	51
7.3.4. Logika tury: specjalizacja i konwersja zasobów	52
7.3.5. Odrzucanie kart i mechanizm awaryjny	52
7.3.6. Mechanizm awaryjny: powrót do bota zbalansowanego	52
7.3.7. Bot celujący w karty rozwoju	52
7.3.8. Ustawienie początkowe i logika tury	53
7.3.9. Polityka odrzucania i ograniczenia strategii	53
7.3.10. Bot celujący w najdłuższą drogę	53
7.3.11. Ustawienie początkowe i logika tury	54
7.3.12. Polityka odrzucania i ograniczenia strategii	54
7.4. Boty oparte na algorytmie genetycznym	54
8. Eksperymenty i analiza wyników	59
8.1. Metodologia porównania botów	59
8.1.1. Metryki oceny skuteczności	59

8.2. Scenariusze testowe	60
8.2.1. Scenariusz 2: Analiza iteracyjnego rozwoju heurystyk	64
9. Podsumowanie i wnioski	81
9.1. Ocena realizacji celów pracy	81
9.2. Wnioski z części badawczej	81
9.3. Możliwości dalszego rozwoju systemu	82
Bibliografia	83

Rozdział 1.

Wstęp

1.1. Wprowadzenie do tematyki gier planszowych i sztucznej inteligencji

Gry planszowe stanowią jedną z najstarszych form rywalizacji intelektualnej i strategicznej pomiędzy ludźmi, sięgającą czasów starożytnych. Od czasów szachów, poprzez warcaby, aż po współczesne gry eurystyczne takie jak *Catan*, *Ticket to Ride* czy *Agricola*, gry planszowe ewoluowały w kierunku coraz większej złożoności strategicznej i głębi mechaniki gameplayowej. Współczesne gry planszowe łączą w sobie elementy takie jak zarządzanie zasobami, podejmowanie decyzji w warunkach niepełnej informacji, interakcję wieloosobową oraz wpływ losowości na przebieg rozgrywki. Te cechy sprawiają, że gry planszowe stanowią doskonałe laboratorium do badania algorytmów decyzyjnych, strategii adaptacyjnych oraz zachowań konkurencyjnych w sformalizowanych środowiskach o mniej lub bardziej przewidywalnych regułach.

Sztuczna inteligencja w kontekście gier planszowych od dziesięcioleci przyciąga uwagę badaczy zajmujących się teorią gier, algorytmiką oraz uczeniem maszynowym. Klasyczne podejścia, takie jak algorytm Minimax czy alfa-beta pruning, znalazły zastosowanie w grach deterministycznych takich jak szachy. Dla gier o większej złożoności stanowej, wyższej losowości lub elementach niepełnej informacji opracowano zaawansowane metody, takie jak Monte Carlo Tree Search (MCTS). Automatyczne boty umożliwiają symulowanie tysięcy rozgrywek w krótkim czasie, analizę drzew decyzyjnych oraz ocenę skuteczności poszczególnych strategii. Dzięki temu możliwe jest identyfikowanie niezbalansowanych mechanik, dominujących sposobów gry oraz rzadkich, lecz istotnych scenariuszy, które mogłyby zostać pominięte w testach manualnych. W efekcie podejście to prowadzi do bardziej obiektywnej analizy systemu gry i usprawnia proces iteracyjnego doskonalenia zasad.

1.2. Gra Catan – charakterystyka i potencjał badawczy

Catan (wcześniej znany jako *The Settlers of Catan*) to strategiczna gra planszowa zaprojektowana przez Klausa Teubera, w której gracze rozwijają osadnictwo na wyspie poprzez pozyskiwanie surowców, handel oraz budowę dróg, osad i miast. Rozgrywka łączy elementy planowania ekonomicznego z umiarkowaną losowością wynikającą z rzutów kośćmi decydujących o produkcji zasobów.

1.3. Plansza, cel gry i podstawowe zasady

Plansza gry *Catan* składa się z heksagonalnych pól reprezentujących różne typy terenu, takich jak lasy, wzgórze, pola uprawne, pastwiska oraz góry, z których każde odpowiada określonemu rodzajowi surowca. Pola te rozmieszczone są w sposób modularny, co powoduje, że każda rozgrywka posiada inną konfigurację przestrzenną. Pomiędzy heksami znajdują się węzły (skrzyżowania), na których gracze mogą budować osady i miasta, oraz krawędzie, na których budowane są drogi.

Każdy typ pola produkcyjnego na planszy odpowiada określonemu surowcowi:

- lasy produkują drewno (*Lumber*),
- wzgórze produkują cegłę (*Brick*),
- pola uprawne produkują zboże (*Grain*),
- pastwiska produkują wełnę (*Wool*),
- góry produkują rudę (*Ore*).

Każdy heks produkcyjny posiada przypisaną losowo liczbę z zakresu 2–12 (z wyjątkiem 7), która odpowiada możliwym sumom wyrzuconym na dwóch sześciennych kościach. Liczby te determinują, kiedy dany heks produkuje surowce — po rzucie kośćmi wszystkie hekсы oznaczone liczbą odpowiadającą wyrzuconej sumie generują zasoby dla graczy posiadających przy nich osady lub miasta [7]. Pola pustynne nie generują zasobów i stanowią początkową lokalizację rozbójnika.

Rozbójnik to specjalna figura, która blokuje produkcję surowców z heksu, na którym się znajduje. Gdy gracz wyrzuci sumę 7 na kościach, musi przemieścić rozbójnika na wybrany heks (blokując jego produkcję) i może ukraść jedną kartę zasobów od gracza posiadającego osadę lub miasto przy tym heksie.

Celem gry jest zdobycie **10 punktów zwycięstwa** (15 w wariacie rozgrywki 1 vs 1). Punkty te są przyznawane głównie za budowę osad (1 punkt zwycięstwa) i miast (2 punkty zwycięstwa).

Osady i miasta muszą być połączone drogami — każda nowa struktura musi być dostępna przez ciągłą sieć dróg gracza. Ponadto obowiązuje **zasada dystansu**:



Rysunek 1.1: Przykładowa plansza gry Catan

osady i miasta różnych graczy (oraz własne) muszą być oddalone od siebie o co najmniej dwie krawędzie.

Każda struktura ma określony koszt budowy wyrażony w surowcach:

- **Droga:** drewno + cegła (1+1),
- **Osada:** drewno + cegła + wełna + zboże (1+1+1+1),
- **Miasto:** ruda + ruda + ruda + zboże + zboże (3+2) — budowane poprzez modernizację istniejącej osady,
- **Karta rozwoju:** ruda + wełna + zboże (1+1+1).

Karty rozwoju stanowią dodatkowy element strategiczny gry. Po zakupie karty są losowane z talii i mogą być trzech typów: **Rycerz** (umożliwia przemieszczenie rozbójnika i kradzież zasobu od przeciwnika), **Postęp** (karty specjalne takie jak Monopol, Rok Obfitości czy Budowa Dróg) oraz **Punkty zwycięstwa** (ukryte karty dające 1 VP każda, ujawniane dopiero przy osiągnięciu zwycięstwa). Karty Rycerz mogą być zagrywane w dowolnym momencie tury, natomiast karty Postępu i Punkty zwycięstwa są zagrywane przed rzutem kośćmi.

Dodatkowymi źródłami punktów zwycięstwa są premie:



Rysunek 1.2: Koszty budowy struktur w grze Catan

- **Premia Najdłuższa Droga.** Przyznawana jest graczowi posiadającemu najdłuższy nieprzerwany ciąg połączonych dróg o długości co najmniej pięciu segmentów. Premia ta może zostać odebrana przez innego gracza, jeśli zbuduje on dłuższą drogę.
- **Premia Największa Armia.** Przyznawana jest graczowi, który jako pierwszy użyje co najmniej trzech kart Rycerz i posiada ich więcej niż pozostali gracze. Podobnie jak w przypadku Najdłuższej Drogi, premia ta jest dynamiczna i może przechodzić pomiędzy graczami w trakcie rozgrywki.

Każda z tych premii zapewnia dodatkowe **2 punkty zwycięstwa**, istotnie wpływając na strategię oraz tempo gry. W dalszej części pracy punkty zwycięstwa będą oznaczane skrótem **VP** (ang. *Victory Points*). Rozgrywka toczy się w turach, a zwycięstwo następuje natychmiast po osiągnięciu wymaganej liczby punktów przez

jednego z graczy.

Podstawowy przebieg tury obejmuje rzut dwiema kośćmi sześciennymi, który determinuje produkcję zasobów na planszy. Gracze otrzymują surowce z tych pól, których numer odpowiada wyrzuconej sumie — osada zapewnia 1 zasób z każdego przylegającego heksu, a miasto 2 zasoby. W przypadku gdy została wyrzucona liczba 7, żaden heks nie produkuje zasobu. Zamiast tego gracze posiadające więcej niż 9 kart zasobów (zgodnie z regułami wariantu 1 vs 1) muszą odrzucić połowę (zaokrąglając w dół), a aktywny gracz przemieszcza rozbójnika na wybrany heks (blokując jego produkcję) i może ukraść jedną kartę zasobów od gracza posiadającego osadę lub miasto przy tym heksie [7].

Następnie możliwe jest wykonywanie akcji budowy, takich jak wznoszenie dróg, osad, miast lub zakup kart rozwoju. W klasycznej wersji gry występuje również handel (z innymi graczami lub z bankiem). Handlowanie z bankiem odbywa się według standardowego kursu 4:1 (cztery dowolne zasoby za jeden wybrany), jednak gracze posiadające osadę lub miasto przy porcie mogą korzystać z lepszych kursów: port 3:1 (trzy dowolne za jeden) lub port 2:1 (dwa konkretne surowce za jeden tego samego typu).

Istotnym elementem gry jest losowość wynikająca z rzutów kośćmi, która wpływa na tempo pozyskiwania zasobów, jednak decyzje strategiczne — wybór lokalizacji budowy, kierunek rozwoju infrastruktury oraz zarządzanie zasobami — mają kluczowe znaczenie dla długoterminowego sukcesu. Ta kombinacja **niepełnej informacji, losowości i planowania** sprawia, że *Catan* stanowi interesujące środowisko badawcze dla analizy algorytmów decyzyjnych i strategii gry.

1.4. Produkcja zasobów i miara pipsów

Jak wspomniano wcześniej, każde pole produkcyjne (heks) na planszy posiada przypisaną liczbę z zakresu 2–12 (z wyjątkiem 7). Ze względu na różną liczbę kombinacji prowadzących do danej sumy, poszczególne liczby mają odmienne prawdopodobieństwo wystąpienia. W praktyce wprowadza się miarę **pipsów** (z ang. *pips*, dosłownie: oczka na kości), która odzwierciedla częstość występowania danej liczby:

- **2 lub 12:** 1 pips (1 kombinacja: 1+1 lub 6+6),
- **3 lub 11:** 2 pipsy (2 kombinacje: 1+2, 2+1 lub 5+6, 6+5),
- **4 lub 10:** 3 pipsy (3 kombinacje),
- **5 lub 9:** 4 pipsy (4 kombinacje),
- **6 lub 8:** 5 pipsów (5 kombinacji).

Miara pipsów stanowi użyteczne narzędzie analityczne, ponieważ bezpośrednio odzwierciedla **wartość oczekiwaną produkcji** z danego pola. Heksy o wyższych pipsach generują zasoby częściej, co czyni je bardziej wartościowymi celami w fazie ustawień początkowych oraz podczas oceny potencjału produkcyjnego pozycji gracza. Pojęcie to będzie wykorzystywane w dalszej części pracy przy opisie strategii botów, które oceniają jakość lokalizacji budowy oraz prognozują przyszłą produkcję zasobów. W szerszej perspektywie pipsy stanowią również praktyczną miarę ograniczającą wpływ losowości na długoterminowy wynik rozgrywki, co jest często omawiane w analizach relacji „luck vs skill” w Catanie [10].

1.5. Wariant 1 vs 1 przyjęty w pracy

W niniejszej pracy analizowana jest wersja gry *Catan* w wariantcie 1 vs 1. W wariantcie tym pominięto handel pomiędzy graczami, ponieważ w rozgrywce dwuosobowej traci on swój negocjacyjny charakter, a często sprowadza się do decyzji trywialnych lub symetrycznych.

Pominięcie handlu między graczami upraszcza część społeczno-negocjacyjną gry, jednocześnie zwiększając kontrolę eksperymentalną — umożliwia prowadzenie powtarzalnych symulacji, jednoznaczne porównywanie botów oraz przypisywanie obserwowanych różnic w wynikach konkretnym strategiom decyzyjnym. Jednocześnie w wariantcie 1 vs 1 zachowane zostają kluczowe własności istotne z punktu widzenia badań nad algorytmami decyzyjnymi, takie jak zarządzanie zasobami, decyzje przestrzenne na planszy, losowość wynikająca z rzutów kośćmi oraz bezpośrednia rywalizacja o punkty zwycięstwa i osiągnięcia specjalne.

1.6. Zalety gry *Catan*

Jedną z najczęściej wskazywanych zalet gry *Catan* jest umiejętne połączenie relatywnie prostych zasad z wysokim poziomem satysfakcji oraz głębią strategiczną. W recenzji opublikowanej na łamach magazynu *Pyramid* podkreślono, że gra oferuje satysfakcjonujące doświadczenie rozwoju ekonomicznego i wymiany zasobów, przy jednocześnie umiarkowanym czasie rozgrywki, wynoszącym zazwyczaj od półtorej do dwóch godzin. Zwrócono uwagę, że poziom satysfakcji płynący z rozgrywki jest porównywalny z grami o znacznie dłuższym czasie trwania.

Drugim, bardzo istotnym atutem jest wysoka regrywalność: plansza składa się z heksów, a ich układ i przypisane numery produkcji można zmieniać między partiami, co ogranicza powtarzalność i utrudnia wyuczenie jednej „sztywnej” sekwencji optymalnych ruchów. Dzięki temu gra sprzyja analizie adaptacyjnych strategii oraz reagowania na bieżącą sytuację na planszy.

Catan jest uznanym tytułem o silnym wpływie kulturowym i branżowym: zdobył prestiżowe nagrody (m.in. Spiel des Jahres w 1995 oraz Game of the Century według Gamescom w 2015) i stał się jednym z symboli „nowoczesnych” gier planszowych [9].

1.6.1. Scena turniejowa i mistrzostwa

Istotnym potwierdzeniem dojrzałości gry jako dyscypliny rywalizacyjnej jest rozbudowana scena turniejowa. Oficjalna strona CATAN opisuje system mistrzostw obejmujący turnieje krajowe oraz wydarzenia międzynarodowe, takie jak mistrzostwa świata, Europy czy obu Ameryk [8].

Równolegle funkcjonuje program „CATAN Championship” organizowany w wielu krajach: gracze rywalizują w turniejach kwalifikacyjnych i narodowych, a zwycięzcy uzyskują możliwość udziału w wydarzeniach najwyższej rangi. Opis tej struktury (rundy wstępne, przejście do fazy pucharowej, awans do finałów narodowych) pokazuje, że *Catan* posiada standaryzowane ramy rywalizacji, co jest korzystne także z perspektywy projektowania eksperymentów porównawczych z udziałem botów.

1.6.2. Potencjał badawczy (w kontekście botów)

Z perspektywy badań nad algorytmami decyzyjnymi *Catan* jest szczególnie interesujący, ponieważ łączy kilka źródeł złożoności:

- **losowość** (dystrybucja zasobów zależna od rzutów),
- **wieloetapowe decyzje sekwencyjne** (budowa, handel, rozwój),
- **interakcję strategiczną** (blokady, wyścig o przestrzeń i punkty),
- **wiele dróg do zwycięstwa** (różne kombinacje budowy, kart rozwoju i premii).

Połączenie tych elementów sprawia, że *Catan* stanowi wartościowe środowisko do testowania i porównywania różnych strategii algorytmów decyzyjnych.

1.7. Cel i zakres pracy

Celem aplikacyjnym pracy jest zaprojektowanie i implementacja silnika gry *Catan* w wariantcie 1 vs 1 w języku C++, przeznaczonego do wykonywania dużej liczby symulacji oraz do integracji z automatycznymi graczami. Osiągnięcie tego celu wymagało w szczególności opracowania spójnej reprezentacji stanu gry, jednoznacznego systemu akcji oraz mechanizmu deterministycznego wykonywania i cofania ruchów.

Celem badawczym pracy jest zaprojektowanie, implementacja oraz porównanie botów wykorzystujących różne strategie decyzyjne, obejmujące podejścia losowe, heurystyczne oraz przeszukujące. Porównanie przeprowadzono w oparciu o metryki opisujące wyniki i przebieg rozgrywek (m.in. odsetek zwycięstw oraz statystyki zależne od długości gry).

Zakres pracy obejmuje:

- implementację silnika symulacyjnego dla wariantu 1 vs 1 wraz z walidacją reguł,
- implementację zestawu botów oraz wspólnego interfejsu gracza,
- przygotowanie narzędzi do rejestracji i odtwarzania rozgrywek (viewer),
- przeprowadzenie eksperymentów porównawczych i analizę wyników.

1.8. Podział pracy i wkład autorów

Praca została zrealizowana zespołowo. Poniżej przedstawiono podział zadań między autorów.

Maciej Stempniak odpowiadał za:

- implementację silnika gry Catan w wariantcie 1 vs 1, w tym reprezentację stanu gry oraz logikę przebiegu rozgrywki,
- implementację środowiska do przeprowadzania masowych eksperymentów symulacyjnych,
- projekt i implementację wyspecjalizowanych botów: *RoadPlayer*, *DevPlayer* oraz *CityRushPlayer*,
- implementację bota opartego na algorytmie genetycznym,
- opracowanie narzędzia do wizualizacji i odtwarzania przebiegu rozgrywek (viewer).

Yaryna Rachkevych odpowiadała za:

- projekt topologii planszy oraz struktur danych opisujących jej elementy,
- implementację systemu akcji wraz z mechanizmem ich stosowania i cofania (apply/undo),
- projekt i implementację bota wykorzystującego algorytm alpha-beta oraz bota *OneResource*,

- przeprowadzanie eksperymentów porównawczych pomiędzy botami oraz wprowadzenie i analizę metryk skuteczności,
- projekt i implementację testów jednostkowych weryfikujących poprawność działania silnika gry.

Wkład wspólny autorów obejmował:

- definiowanie celów i zakresu pracy,
- projekt oraz implementację botów heurystycznych (iteracje It1–It5),
- przygotowanie części badawczej pracy oraz interpretację wyników symulacji,
- redakcję, weryfikację merytoryczną oraz korektę końcowej wersji pracy.

1.9. Struktura pracy

Rozdział 2 przedstawia przegląd istniejących rozwiązań i prac naukowych związanych z symulacją gry *Catan* oraz projektowaniem botów. Rozdział 3 definiuje założenia wariantu 1 vs 1 oraz wymagania projektowe silnika. W rozdziale 4 opisano architekturę i szczegóły implementacji (model stanu, system akcji, mechanizm cofania oraz narzędzia diagnostyczne). Rozdziały 5–8 prezentują sposób uruchamiania symulacji, eksperymenty porównawcze, wyniki oraz ich analizę. Rozdział 9 zawiera podsumowanie oraz wnioski.

Rozdział 2.

Przegląd istniejących rozwiązań

2.1. Istniejące implementacje Catana

Gra *Settlers of Catan* od wielu lat stanowi obiekt zainteresowania zarówno hobbystycznych projektów programistycznych, jak i badań naukowych nad sztuczną inteligencją w grach planszowych. W związku z tym powstało kilka implementacji silników gry oraz środowisk symulacyjnych, które umożliwiają eksperymentowanie z automatycznymi graczami.

2.1.1. Catanatron

Catanatron jest jednym z najbardziej zaawansowanych otwartych środowisk symulacyjnych dla gry *Settlers of Catan*. Projekt został zaprojektowany z myślą o masowym uruchamianiu symulacji oraz testowaniu algorytmów decyzyjnych, w szczególności metod opartych na uczeniu przez wzmacnianie. Architektura systemu umożliwia łatwą integrację własnych agentów poprzez zunifikowany interfejs gracza. Dodatkowo środowisko udostępnia wrapper kompatybilny z OpenAI Gym, co pozwala na wykorzystanie istniejących bibliotek RL. [1].

2.1.2. JSettlers

JSettlers to starszy, lecz istotny projekt implementujący grę Catan w języku Java. System zawiera zarówno silnik gry, jak i proste boty regułowe. Projekt ten był wielokrotnie wykorzystywany jako punkt wyjścia w pracach badawczych i dydaktycznych, szczególnie w kontekście gier wieloagentowych. [2].

2.1.3. CatanAI i projekty eksperymentalne

Istnieją również projekty badawcze i studenckie, takie jak *CatanAI*, implementujące grę w Pythonie wraz z agentami opartymi o heurystyki, Reinforcement Learning lub Monte Carlo Tree Search. Projekty te często nie są zoptymalizowane wydajnościowo, lecz służą jako środowiska eksperymentalne do testowania algorytmów. [3].

2.1.4. Tabela porównawcza istniejących rozwiązań

Projekt / Praca	Język	Typ	Obsługa botów	Zastosowanie
Catanatron	Python	Symulator	RL, heurystyki	Badania, benchmarki
JSettlers	Java	Silnik gry	Regułowe	Edukacja
CatanAI	Python	Framework AI	RL, MCTS	Eksperymenty
Szita et al. (2009)	—	Publikacja	MCTS	Badania naukowe
Burre et al. (2020)	—	Publikacja	Deep RL	Badania naukowe

Tabela 2.1: Tabela porównawcza istniejących rozwiązań

2.2. Boty i algorytmy decyzyjne w grach planszowych

Projektowanie botów do gier planszowych stanowi klasyczny problem w dziedzinie sztucznej inteligencji. W zależności od złożoności gry oraz dostępnej informacji stosuje się różne podejścia decyzyjne.

2.2.1. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) jest algorytmem przeszukiwania drzewa gry opartym na losowych symulacjach. Algorytm ten odniósł znaczący sukces w grach o dużej przestrzeni stanów, takich jak Go. Jego zastosowanie w grach z losowością i wieloma graczami, w tym w *Settlers of Catan*, zostało opisane w literaturze naukowej.

Szita, Chaslot i Spronck zaprezentowali jedną z pierwszych analiz zastosowania MCTS w Catanie, wskazując na możliwość adaptacji algorytmu do środowisk z niepełną informacją i elementami losowymi [4].

2.2.2. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (Reinforcement Learning) umożliwia agentom nabywanie strategii poprzez interakcję z otoczeniem i maksymalizację funkcji nagrody.

W kontekście Catana badano zarówno klasyczne metody RL, jak i ich połączenie z MCTS.

W pracy Burre i in. przedstawiono zastosowanie algorytmów Deep Q-Learning w środowisku Catana, analizując skuteczność agentów uczonych przez symulacje [5]. Inne badania skupiają się na problemie negocjacji i handlu pomiędzy agentami, co stanowi istotny element rozgrywki w Catanie [6].

2.2.3. Heurystyki i podejścia regułowe

Alternatywą dla metod uczących się są boty oparte na heurystykach eksperckich. Strategie te wykorzystują wiedzę domenową, taką jak wartości pól, prawdopodobieństwa rzutów kośćmi czy priorytety budowy. Heurystyki te są często stosowane jako punkty odniesienia (baseline) w eksperymentach porównawczych.

2.3. Podsumowanie i uzasadnienie autorskiego podejścia

Przegląd istniejących rozwiązań wskazuje, że dominujące implementacje środowisk dla *Catana* (oraz gier o podobnym charakterze) można w uproszczeniu podzielić na dwie grupy. Pierwsza z nich koncentruje się na kompletności funkcjonalnej i ergonomii (często w językach wysokiego poziomu), co sprzyja szybkiemu prototypowaniu, lecz utrudnia uruchamianie dużej liczby symulacji w krótkim czasie. Druga grupa skupia się na wydajności i możliwości integracji z algorytmami decyzyjnymi, jednak często odbywa się to kosztem przejrzystości stanu, jednoznaczności reguł lub możliwości wygodnej diagnostyki przebiegu gry.

W niniejszej pracy przyjęto autorskie podejście łączące oba cele: (1) budowę możliwie wydajnego i testowalnego silnika gry w wariantcie 1 vs 1 oraz (2) stworzenie infrastruktury eksperymentalnej do porównywania botów o różnych strategiach, wraz z narzędziami umożliwiającymi weryfikację i analizę rozgrywek.

Podstawowym uzasadnieniem przyjętych decyzji projektowych jest charakter zadania badawczego. Porównywanie botów wymaga uruchamiania tysięcy gier, a w konsekwencji powtarzalnego:

- generowania akcji legalnych w danym stanie,
- wykonywania akcji zgodnie z regułami,
- oceny stanu (funkcje heurystyczne),
- oraz (dla podejść przeszukujących) intensywnego symulowania stanów pośrednich.

Z tego powodu kluczowe stały się wymagania нефunkcjonalne: szybkość symulacji, deterministyczność operacji na stanie oraz możliwość ścisłej walidacji poprawności reguł.

2.3.1. Uzasadnienie wyborów projektowych

Wariant 1 vs 1 jako kompromis badawczy. W pracy zastosowano wariant rozgrywki dwuosobowej, w którym pominięto handel pomiędzy graczami, pozostawiając handel z bankiem oraz mechanikę portów. Uzasadnieniem jest zwiększenie kontroli eksperymentalnej: eliminacja negocjacji ogranicza elementy trudne do jednoznacznego modelowania i powtórzenia w symulacjach, a jednocześnie zachowuje kluczowe mechaniki decyzyjne: zarządzanie zasobami, decyzje przestrzenne (drogi/osady/miasta), losowość wynikającą z rzutów kośćmi oraz interakcję poprzez rozbójnika i rywalizację o premie.

Pakowana reprezentacja stanu gry. Rdzeń systemu stanowi struktura `Board::BoardState`, która przechowuje kompletny stan rozgrywki (heksy, węzły, krawędzie, bank oraz gracze). Zastosowano pakowanie informacji w typach liczbowych (`uint16_t`, `uint32_t`, `uint64_t`) m.in. dla:

- heksów (liczba, zasób, liczniki produkcji dla obu graczy),
- węzłów (typ budowli, właściciel, sąsiedztwo heksów i krawędzi, port),
- krawędzi (droga, właściciel, węzły końcowe),
- stanu gracza i banku (zasoby, karty rozwoju, flagi najdłuższej drogi/największej armii, dostępne budowle).

Takie podejście ma dwa bezpośrednie uzasadnienia. Po pierwsze, redukuje narzut pamięci i poprawia lokalność danych, co jest istotne przy masowym uruchamianiu symulacji. Po drugie, umożliwia bardzo szybkie kopiowanie stanu — co wprost wspiera boty wykorzystujące symulacje na kopiach `BoardState` (np. w przeszukiwaniu alfa-beta).

Jednolita, pakowana reprezentacja akcji. Każda decyzja w grze jest modelowana jako `Action::PackedAction` (64-bit), zawierająca typ akcji, identyfikator gracza, argumenty oraz (w razie potrzeby) wektor zasobów. Ujednolicenie formatu akcji pozwala traktować akcję jako „dane” i budować wokół niej:

- jeden interfejs decyzyjny botów,
- wspólny mechanizm wykonania i cofania,
- oraz spójny zapis do logów.

W praktyce jest to realizacja idei wzorca *Command*: boty generują opis operacji, natomiast wykonanie (i walidacja) jest skupione w silniku [13].

Mechanizm apply/undo jako fundament testowalności i AI. `Board::BoardState` udostępnia metody `applyAction()` oraz `undoLastAction()` i utrzymuje własną kolejkę `actionQueue`, co umożliwia cofanie zmian stanu. W warstwie testów wykorzystano to do weryfikacji, że dla reprezentatywnego zbioru akcji wykonanie i cofnięcie przywraca stan do wartości wyjściowej. Z perspektywy botów mechanizm ten jest również kluczowy, ponieważ pozwala realizować „symulację w przód” i analizę konsekwencji decyzji.

Generowanie akcji legalnych w jednym miejscu. Silnik udostępnia zestaw funkcji generujących akcje możliwe w danym stanie (np. budowa drogi/osady/miasta, handel z bankiem, zakup i zagrywanie kart rozwoju, akcje fazy początkowej). Dzięki temu boty nie implementują reguł gry w swoich klasach (co prowadziłoby do powielania logiki i ryzyka rozbieżności), lecz wybierają spośród akcji wygenerowanych przez silnik. Jest to uzasadnione zarówno testowalnością (reguły w jednym miejscu), jak i poprawnością eksperymentów (każdy bot działa w tym samym zbiorze dopuszczalnych decyzji).

Rozdzielenie silnika i strategii graczy przez interfejs `IPlayer`. Boty implementują spójny interfejs decyzji dla poszczególnych etapów tury (`getInitialPlacement()`, `get2InitialPlacement()`, `getDevAction()`, `getDiscardAction()`, `getMoveRobber()`, `getTurnAction()`). Pozwala to:

- porównywać strategie przy identycznym „kontrakcie” wejścia/wyjścia,
- wymieniać boty bez zmian w silniku,
- uruchamiać eksperymenty z użyciem jednego programu uruchomieniowego.

Istotnym elementem bezpieczeństwa architektury jest dodatkowa weryfikacja w `Game`: wywołania metod botów są „guardowane” przez porównanie kopii stanu sprzed i po wywołaniu. Dzięki temu bot nie może modyfikować stanu gry bezpośrednio, a jedyną drogą zmiany stanu jest przekazanie akcji do silnika. Chroni to przed trudnymi do wykrycia błędami i zwiększa wiarygodność eksperymentów.

Dobór botów i iteracyjny rozwój heurystyk. W projekcie zaimplementowano kilka klas botów, które pełnią rolę punktów odniesienia i kolejnych ulepszeń:

- bot losowy (`RandomPlayer`) jako baseline,
- boty heurystyczne rozwijane iteracyjnie (`It1–It5`),
- boty specjalizowane (np. ukierunkowane na karty rozwoju lub drogi),
- bot przeszukujący (`alphaBetaPlayer`), wykorzystujący symulację na kopiach stanu oraz alfa-beta pruning.

Takie spektrum botów uzasadnia się potrzebą uzyskania zarówno prostych punktów odniesienia, jak i bardziej zaawansowanych strategii, które wykorzystują strukturę problemu. W szczególności `alphaBetaPlayer` pokazuje, że zaproponowany model stanu i akcji jest wystarczająco wydajny, aby umożliwiać przeszukiwanie drzewa decyzji w rozsądnych limitach (ograniczanie gałęziowania poprzez filtrowanie akcji deterministycznych oraz ograniczanie liczby kandydatów).

Parametryzacja heurystyk i trening. W celu oddzielenia „pomysłu na ocenę” od doboru konkretnych wag, zaimplementowano gracza parametryzowanego (`ParaPlayer`), który wczytuje współczynniki z pliku konfiguracyjnego. Umożliwia to automatyzację strojenia (np. poprzez skrypt uruchamiający serie gier i selekcję parametrów na podstawie win-rate), a także ułatwia raportowanie eksperymentów: zmiana strategii może być opisana zmianą konfiguracji, bez modyfikacji kodu źródłowego.

Diagnostyka: rejestracja i odtwarzanie rozgrywek. Dla potrzeb analizy oraz debugowania opracowano moduł `Dumper`, zapisujący przebieg gry do formatu JSONL (stan początkowy, rzuty kośćmi, zastosowane akcje, stany na początku i końcu tury). Uzupełnieniem jest narzędzie `utils/replay_viewer.py`, które potrafi odtworzyć i zwizualizować rozgrywkę na podstawie logu. Uzasadnieniem tego elementu jest praktyczna obserwacja, że w projektach łączących złożoną logikę reguł i boty decyzyjne sama poprawność testów nie wystarcza — konieczne są narzędzia pozwalające prześledzić przebieg rozgrywki i szybko zidentyfikować, czy błąd wynika z silnika, czy z polityki decyzyjnej.

Walidacja poprawności poprzez testy. Silnik został objęty zestawem testów jednostkowych i integracyjnych opartych o `GoogleTest`. Szczególną rolę pełnią testy weryfikujące poprawność generowania akcji oraz mechanizmu cofania (`undoLastAction`) dla różnych typów akcji. Jest to bezpośrednio uzasadnione tym, że nawet subtelne błędy reguł mogłyby przekładać się na błędne wnioski w części porównawczej botów.

2.3.2. Podsumowanie

Autorskie podejście w pracy można streścić jako budowę środowiska symulacyjnego, które świadomie faworyzuje cechy istotne dla badań porównawczych botów: wydajność, deterministyczność operacji na stanie, jedno źródło prawdy dla reguł (generator akcji legalnych + wykonanie akcji), a także silną testowalność i możliwość diagnostyki przebiegu gry (logi JSONL i odtwarzacz). W rezultacie otrzymano spójny system, w którym silnik jest niezależny od strategii, a strategię mogą być rozwijane (heurystycznie, parametrycznie i przeszukująco) bez naruszania poprawności i spójności reguł gry.

Rozdział 3.

Analiza problemu i założenia projektowe

3.1. Wymagania funkcjonalne gry

Celem implementacji jest zapewnienie kompletnego, spójnego przebiegu rozgrywki w wariantcie *Catan* 1 vs 1, tak aby stan gry mógł być wykorzystywany zarówno do pojedynczych uruchomień (debugowanie i demonstracje), jak i do masowych eksperymentów porównujących boty. Wymagania funkcjonalne wynikają z konieczności odwzorowania kluczowych mechanik gry oraz z potrzeby udostępnienia jednoznacznych punktów decyzyjnych dla graczy automatycznych.

W ramach niniejszej pracy system powinien realizować następujące funkcje:

1. Inicjalizacja i generowanie planszy

- wygenerowanie losowej konfiguracji planszy przed rozpoczęciem gry (rozmieszczenie typów zasobów i numerów produkcji),
- ustawienie początkowej pozycji rozbójnika,
- zainicjalizowanie stanu banku i talii kart rozwoju.

2. Obsługa fazy początkowej (initial placement)

- realizacja standardowej kolejności „węża” dla dwóch graczy: Player0, Player1, Player0, Player1, Player0,
- umożliwienie każdemu graczowi wyboru osady i drogi w ramach dozwolonych lokalizacji,
- przyznanie zasobów startowych wynikających z drugiej osady (zgodnie z implementacją).

3. Obsługa pełnego cyklu tury

- obsługa zagrywania kart rozwoju w dedykowanej fazie (przed i po rzucie kośćmi),
- wykonanie rzutu dwiema kośćmi i produkcja zasobów zgodnie z numerami na heksach,
- obsługa przypadku wyrzucenia 7: odrzucanie zasobów oraz faza rozbójnika,
- umożliwienie wykonywania akcji w turze aż do zakończenia tury przez akcję kończącą.

4. Realizacja katalogu akcji zgodnych z zasadami

- budowa dróg, osad i miast przy zachowaniu ograniczeń (m.in. zasada dystansu, wymaganie połączenia drogą),
- handel z bankiem w kursie 4:1 oraz obsługa portów 3:1 i 2:1,
- zakup kart rozwoju i zagrywanie kart typu: Rycerz (wraz z ruchem rozbójnika), Budowa Dróg, Rok Obfitości, Monopol,
- utrzymywanie i aktualizacja premii (najdłuższa droga / największa armia) oraz ich wpływ na wynik.

5. Warunek zakończenia gry i raportowanie wyniku

- zakończenie rozgrywki po osiągnięciu progu punktów zwycięstwa w wariantcie 1 vs 1 (w implementacji: 15 VP po uwzględnieniu premii),
- limit maksymalnej liczby tur jako zabezpieczenie przed nieskończoną rozgrywką,
- możliwość wielokrotnego uruchamiania gier (batch) z raportowaniem statystyk.

6. Interfejs dla botów i narzędzia wspierające eksperymenty

- spójny interfejs `IPlayer` pozwalający botom podejmować decyzje w punktach decyzyjnych silnika,
- rejestrowanie przebiegu gry do logów (format JSONL) oraz możliwość odtwarzania gry w narzędziu wizualizacyjnym.

3.2. Wymagania niefunkcjonalne (wydajność, testowalność)

W części badawczej pracy kluczowe jest uruchamianie dużej liczby symulacji, a także (dla bardziej zaawansowanych botów) intensywne symulowanie stanów pośrednich w trakcie pojedynczej tury. Stąd wynikają następujące wymagania niefunkcjonalne.

Wydajność symulacji.

- stan gry powinien być reprezentowany w sposób zwarty, sprzyjający lokalności danych i szybkiemu kopiowaniu,
- operacje wykonywania ruchu oraz generowania akcji legalnych muszą być wystarczająco szybkie, aby umożliwiać eksperymenty na tysiącach gier,
- losowość powinna być generowana niskokosztowo, a zarazem umożliwiać izolowanie symulacji pomocniczych (np. w przeszukiwaniu) od losowości „właściwej” gry.

Deterministyczność i odtwarzalność.

- dla ustalonego stanu wejściowego i konkretnej akcji wynik zastosowania akcji powinien być jednoznaczny,
- mechanizm cofania powinien deterministycznie przywracać stan do wartości sprzed zastosowania akcji,
- logi powinny zawierać informacje umożliwiające analizę przebiegu gry krok po kroku.

Testowalność i weryfikacja poprawności reguł.

- logika reguł powinna być możliwa do testowania w izolacji od botów,
- kluczowe mechanizmy (w szczególności `applyAction()` / `undoLastAction()` oraz generowanie akcji) muszą być pokryte testami regresyjnymi,
- błędy powinny być możliwie szybko lokalizowane; stąd istotna jest diagnostyka (logi, odtwarzacz) oraz mechanizmy ochronne.

Separacja odpowiedzialności i bezpieczeństwo interfejsu botów.

- boty nie mogą modyfikować stanu gry bezpośrednio — jedynym sposobem zmiany stanu jest wykonanie akcji przez silnik,

- system powinien wykrywać próby naruszenia tej zasady w trakcie działania (np. poprzez porównanie stanu przed i po wywołaniu metody bota).

Powyższe wymagania uzasadniają kluczową zasadę projektową przyjętą w pracy: **jedno źródło prawdy dla reguł** — boty operują wyłącznie na stanie i zbiorze akcji wygenerowanych przez silnik, a interpretacja i wykonanie akcji odbywa się centralnie w jednej implementacji.

3.3. Model stanu gry i akcji

Model domenowy zaprojektowano tak, aby stan gry był możliwie zwarty oraz aby operacje na nim były jednoznaczne i testowalne. Rdzeń stanowi struktura `Board::BoardState`, a wszystkie decyzje i zdarzenia w grze są kodowane jako `Action::PackedAction`.

3.3.1. Stan gry

`Board::BoardState` zawiera m.in.:

- pozycję rozbójnika (`robberPosition`),
- tablice heksów, węzłów i krawędzi o stałym rozmiarze (topologia planszy jest stała, a zmienia się jedynie ich zawartość),
- pakowany stan dwóch graczy (`packedPlayers`) oraz banku (`packedBank`),
- informacje sterujące przebiegiem gry (`currentPlayer`, `currentTurn`),
- historię zastosowanych akcji (`actionQueue`) wykorzystywaną do cofania.

Szczególnie istotna jest decyzja, aby heksy przechowywały nie tylko typ zasobu i numer, lecz także zakodowaną informację o „udziale w produkcji” dla obu graczy. Dzięki temu produkcja po rzucie kośćmi może być realizowana bez analizowania sąsiedztwa węzłów w każdej turze.

3.3.2. Akcja jako nośnik decyzji

Akcja jest kodowana jako 64-bitowa wartość zawierająca:

- typ akcji (np. budowa, handel, rzut kośćmi, koniec tury),
- identyfikator gracza wykonującego akcję,
- argumenty liczbowe (np. identyfikator krawędzi/węzła/heksu, typ zasobu, kurs portu),

- opcjonalny wektor zasobów (np. w odrzucaniu zasobów).

Jednolity format akcji ma kluczowe znaczenie dla spójności systemu: boty zwracają akcje, silnik je interpretuje, a ten sam zapis może zostać wykorzystany do logowania i odtwarzania.

3.3.3. „Jedno źródło prawdy” dla reguł

W systemie przyjęto zasadę, że **reguły gry są implementowane wyłącznie w silniku**, a boty nie powielają logiki walidacji. Z technicznego punktu widzenia składają się na to dwa elementy:

1. **Generowanie akcji legalnych** dla danego stanu i gracza (np. `getLegalActions()`) oraz wyspecjalizowane generatory dla budowy, handlu, kart rozwoju i faz początkowych). 2. **Wykonanie akcji** w jednym miejscu (`applyAction()`), które aktualizuje stan zgodnie z regułami oraz zapisuje informację do historii cofania.

W konsekwencji bot działa w modelu: *obserwuj stan* → *wybierz akcję z listy legalnych* → *przełącz do silnika*. Takie podejście minimalizuje ryzyko rozbieżności pomiędzy botami oraz zapewnia porównywalność wyników.

3.3.4. Cofanie akcji i symulacje stanów pośrednich

Mechanizm `undoLastAction()` umożliwia deterministyczne cofanie zmian po zastosowaniu akcji. Jest to istotne zarówno w testach, jak i w botach wykorzystujących symulacje (np. przeszukiwanie alfa-beta), gdzie wielokrotnie analizuje się konsekwencje różnych decyzji na kopiach stanu. W testach regresyjnych weryfikuje się m.in. własność: zastosowanie akcji i jej cofnięcie przywraca stan do wartości początkowej.

Rozdział 4 przedstawia szczegóły implementacyjne powyższych założeń, ze szczególnym uwzględnieniem pakowanej reprezentacji danych, generatorów akcji oraz mechanizmów wspierających diagnostykę i eksperymenty.

Rozdział 4.

Architektura i implementacja systemu

4.1. Wybór technologii i narzędzi

Rozdział 3 opisuje uzasadnienie przyjętych decyzji projektowych. Poniżej przedstawiono technologie i narzędzia w takim zakresie, w jakim są one widoczne w implementacji i procesie budowania projektu.

Projekt jest rozwijany w języku **C++17** (ustawienie standardu w pliku konfiguracyjnym **CMake**). Całość budowana jest z użyciem **CMake**, a zależność do frameworku testowego jest pobierana i konfigurowana automatycznie przez mechanizm **FetchContent**.

Testy zostały zaimplementowane w oparciu o **GoogleTest/GMock** (pobierane jako **v1.14.0**) i uruchamiane przez **CTest** (**gtest_discover_tests**). Dzięki temu testy są wykrywane automatycznie po kompilacji, a pipeline testowy jest spójny niezależnie od platformy.

W projekcie dostępna jest opcjonalna konfiguracja diagnostyczna **ENABLE_ASAN**, która dodaje flagi **AddressSanitizer** przy kompilatorach **GNU/Clang**, co ułatwia wykrywanie błędów pamięci w trakcie rozwoju.

Do analizy przebiegu rozgrywek wykorzystywany jest zapis zdarzeń w formacie **JSONL** (logger w **C++**) [15], a do odtwarzania i wizualizacji logów przygotowano narzędzie w **Python** (interfejs **Tkinter**). Pozwala to odseparować lekki, wydajny runtime symulacji od narzędzi analitycznych.

4.2. Struktura projektu

Struktura repozytorium bezpośrednio odzwierciedla podział na silnik symulacji, implementacje botów, moduł uruchomieniowy oraz testy:

- katalog `game_simulation` zawiera implementację rdzenia gry: stan planszy i jego modyfikacje (`board.*`), generator akcji (`board_generate_actions.cpp`) oraz logikę przebiegu rozgrywki (`game.*`),
- katalog `players/` zawiera wspólny interfejs gracza (`player.hpp`) i konkretne strategie botów (m.in. losowy, iteracyjne heurystyki, alpha-beta, gracze wyspecjalizowani),
- katalog `runs/` zawiera program uruchomieniowy (`run.cpp`) do odpalania symulacji z linii poleceń,
- katalog `utils/` zawiera narzędzia pomocnicze (m.in. logger rozgrywek w JSONL i narzędzia analityczne),
- katalog `tests/` zawiera testy jednostkowe i integracyjne uruchamiane przez CTest.

Z perspektywy systemu budowania istotne są trzy elementy:

- biblioteka `game_simulation`, która kompiluje rdzeń gry, wybrane boty i logger (współdzielony kod dla testów oraz programu uruchomieniowego),
- program `run` (w `runs/`), linkowany z `game_simulation` i rozszerzony o dodatkowe implementacje botów, które mają być dostępne w trybie uruchomieniowym,
- zestaw binarek testowych (w `tests/`), linkowanych z `gtest_main`, `gmock` oraz `game_simulation`.

Logika sterująca przebiegiem gry (fazy, kolejność wywołań API botów) jest zaimplementowana w `Game` (`game_simulation/game.*`), natomiast modyfikacje stanu są skupione w `Board::BoardState` (`game_simulation/board.*`). Decyzje botów są przekazywane do silnika wyłącznie jako wartości typu `Action::PackedAction`.

4.3. Implementacja planszy i elementów gry

Wariant planszy gry jest modelowany jako graf o stałej topologii, jednak w implementacji nie występują dynamiczne struktury grafowe. Zamiast tego zastosowano

tablice o rozmiarach zdefiniowanych w pliku stałych (`consts.hpp`), a powiązania są przechowywane bezpośrednio w pakowanych rekordach elementów planszy.

Stan planszy jest częścią struktury `Board::BoardState` (`game_simulation/board.hpp`) i składa się z trzech tablic:

- `hexes[HEX_COUNT]` (heksy) typu `Board::Hex::PackedHex (uint16_t)`,
- `nodes[NODE_COUNT]` (węzły) typu `Board::Node::PackedNode (uint64_t)`,
- `edges[EDGE_COUNT]` (krawędzie) typu `Board::Edge::PackedEdge (uint32_t)`.

Pakowanie pól zostało opisane jawnie w komentarzach w pliku nagłówkowym i jest realizowane przez zestaw funkcji `pack*/unpack*`.

Heks (`PackedHex`) koduje:

- numer Catana w bitach 0–3,
- typ zasobu w bitach 4–6,
- wartości produkcji przypisane do graczy w bitach 7–12 (po 3 bity na gracza).

Wartości produkcji w heksie nie są „wynikiem rzutu”, lecz licznikami mówiącymi, ile osad/miast danego gracza jest podłączonych do heksu. Dzięki temu w fazie produkcji zasobów (`handleRollDice` w `game_simulation/board.cpp`) możliwe jest rozdzielenie zasobów bez ponownego analizowania sąsiedztw węzłów.

Węzeł (`PackedNode`) koduje:

- typ zabudowy (brak/osada/miasto) oraz właściciela,
- identyfikatory maksymalnie trzech sąsiadujących heksów,
- typ portu,
- identyfikatory maksymalnie trzech sąsiadujących krawędzi.

Krawędź (`PackedEdge`) koduje:

- informację, czy istnieje droga,
- właściciela drogi,
- dwa sąsiadujące węzły.

Stała topologia jest inicjalizowana w konstruktorze `BoardState` poprzez wypełnienie tablic `nodes` (m.in. wskazania sąsiadów oraz portów). W trakcie gry zmieniają się wyłącznie pola związane z zabudową/właścicielem (drogi, osady, miasta) oraz liczniki produkcji w heksach.

4.4. System akcji i cofania ruchów

Wszystkie interakcje botów z silnikiem są ujednolicone do pojedynczego typu `Action::PackedAction(uint64_t)` zdefiniowanego w `game_simulation/actions.hpp`. Akcja jest wartością liczbową, której pola są kodowane bitowo; układ (od najmłodszych bitów) ma postać:

- typ akcji: bity 0–4,
- identyfikator gracza: bity 5–6,
- zasoby (5-bitowe liczniki dla pięciu typów zasobów): bity 7–31,
- argumenty `arg1`, `arg2`, `arg3` (po 8 bitów): bity 32–55.

W pliku nagłówkowym zdefiniowano zestaw funkcji `packType/unpackType`, `packPlayerID/unpackPlayerID`, `packResource/unpackResource` oraz `packArg*/unpackArg*`. W efekcie akcje są łatwe do tworzenia i interpretacji zarówno w botach, jak i w silniku.

Centralnym miejscem wykonania akcji jest metoda `Board::BoardState::applyAction()` (`game_simulation/board.cpp`). Funkcja rozpakowuje typ i identyfikator gracza, a następnie wywołuje odpowiedni handler (`handleBuildRoad`, `handleTradeBank`, `handleRollDice`, itd.). Po wykonaniu akcji jej finalna postać jest dopisywana do historii `actionQueue`.

Istotną cechą implementacji jest to, że w pewnych przypadkach silnik *uzupełnia akcję* o dodatkową informację potrzebną do cofania. Przykłady:

- przy kradzieży zasobu (`StealResource`) do `arg1` zapisywany jest faktycznie skradziony typ zasobu, aby `handleUndoStealResource` mogło odtworzyć zmianę deterministycznie,
- przy budowie drogi i osady (`BuildRoad`, `BuildSettlement`) w `arg2` i `arg3` zapisywane są „meta” pola związane z najdłuższą drogą u obu graczy, tak aby `undo` mogło przywrócić je dokładnie.

Cofanie realizuje metoda `Board::BoardState::undoLastAction()`, która pobiera ostatnią akcję z `actionQueue`, usuwa ją z historii i wywołuje odpowiedni handler `handleUndo*`. W połączeniu z jednoznacznym kodowaniem argumentów pozwala to na szybkie przeszukiwania stanów w botach (np. lookahead lub alfa-beta) oraz na testy własności typu „apply+undo przywraca stan”.

4.5. Zarządzanie stanem gry

Kompletny stan rozgrywki jest przechowywany w strukturze `Board::BoardState` (`game_simulation/board.hpp`). Oprócz planszy (heksy/węzły/krawędzie) zawiera ona m.in.:

- `robberPosition` (aktualne położenie rozbójnika),
- `packedPlayers[2]` (stan dwóch graczy w postaci pakowanej),
- `packedBank` (stan banku),
- `currentPlayer` oraz `currentTurn`,
- `actionQueue` (historia akcji umożliwiająca undo).

Równość stanu (`operator==`) została zdefiniowana jako porównanie wszystkich pól stanu, włącznie z `actionQueue`. Jest to wykorzystywane m.in. jako narzędzie diagnostyczne w warstwie sterującej rozgrywką.

Warstwa `Game` (`game_simulation/game.*`) odpowiada za kolejność faz oraz komunikację z botami. W konstruktorze `Game` boty otrzymują wskaźnik do `boardState`, aby mogły odczytywać stan. Jednocześnie silnik zabezpiecza się przed niepożądaną, bezpośrednią modyfikacją stanu przez bota: wszystkie kluczowe wywołania metod bota są wykonywane przez funkcję osłonową, która kopiuje `BoardState` przed wywołaniem i porównuje go po powrocie. Wykrycie zmiany stanu skutkuje przerwaniem rozgrywki wyjątkiem, co chroni poprawność symulacji.

Opcjonalnie `Game` może zapisywać przebieg rozgrywki przez komponent `Dumper`. Metoda `applyActionLogged()` stosuje akcję do stanu (`BoardState::applyAction`) oraz rejestruje zdarzenie w logu, co umożliwia późniejsze odtworzenie i analizę rozgrywki w narzędziach pomocniczych.

Rozdział 5.

Symulowanie i odtwarzanie rozgrywek

W części badawczej pracy kluczowe jest uruchamianie dużej liczby gier w sposób powtarzalny oraz możliwość analizy przebiegu pojedynczej rozgrywki w przypadku błędów lub interesujących zachowań bota. Z tego powodu opracowano dwa uzupełniające się elementy: program uruchomieniowy do batch-runów oraz moduł rejestracji i odtwarzania rozgrywek.

5.1. Uruchamianie serii rozgrywek

Program `run` (katalog `runs/`) umożliwia uruchamianie serii gier pomiędzy zadanymi botami. W trybie eksperymentalnym istotne są w szczególności:

- uruchamianie wielu gier w jednej sesji (stała liczba gier dla porównywalności wyników),
- redukcja efektu miejsca przez przełączanie stron (`--switch`),
- raportowanie metryk zbiorczych (m.in. win rate, średnia liczba tur, statystyki premii).

5.2. Format rejestracji rozgrywek (JSONL)

Do celów diagnostycznych silnik gry może zapisywać przebieg rozgrywki w formacie **JSONL** (ang. *JSON Lines*), w którym każda linia jest osobnym obiektem JSON opisującym zdarzenie. W logach znajdują się m.in.: stan początkowy, kolejne akcje (w postaci pakowanej), rzuty kośćmi oraz informacje pozwalające odtworzyć przebieg gry krok po kroku.

Wybór JSONL ma uzasadnienie praktyczne: zapis jest strumieniowy, łatwy do przetwarzania skryptami oraz pozwala analizować nawet duże pliki logów bez konieczności wczytywania całości do pamięci.

5.3. Odtwarzanie i wizualizacja (replay viewer)

Do odtwarzania rozgrywek przygotowano narzędzie `utils/replay_viewer.py`, które wczytuje log JSONL i wizualizuje stan gry w kolejnych krokach. Aplikacja została napisana w języku Python z użyciem biblioteki `tkinter`, co pozwala na uruchomienie jej bez zależności zewnętrznych [12].

Odtwarzacz jest wykorzystywany do:

- weryfikacji poprawności implementacji (inspekcja przebiegu gry krok po kroku),
- diagnozowania błędów w botach (np. nieoptymalne wybory lub zapętlenia),
- analizy jakości strategii (np. decyzje o blokowaniu rozbójnikiem, handlu z bankiem i budowie).

5.4. Jak przeprowadzać rozgrywki

Poniżej opisano praktyczny sposób uruchamiania serii gier oraz odtwarzania zapisanych rozgrywek.

Uruchamianie serii gier (program run). Program uruchomieniowy (katalog `runs/`, plik `run.cpp`) uruchamia się z linii poleceń w następujący sposób:

```
run [opcje] <flaga_gracza0> <flaga_gracza1>
```

Argumenty pozycyjne to **flagi botów**: gracz 0 i gracz 1. Dostępne flagi to `m.in` (`RandomPlayer`), `it1–it5` (boty heurystyczne), `para` (`ParaPlayer`), `psit5` (`ParaSettleIt5Player`), `ab` (`AlphaBetaPlayer`), `or` (`OneResourcePlayer`), `dev` (`DevPlayer`), `road` (`RoadPlayer`). Pełną listę oraz opis opcji wyświetla polecenie `run -h` (lub `run --help`).

Ważniejsze opcje:

- `-n N` lub `--games N` – liczba rozgrywek do przeprowadzenia (domyślnie 1);
- `--switch` (lub `--swap`) – przełączanie stron co drugą grę (gracz 0 i gracz 1 zamieniają się miejscami), co redukuje efekt pierwszeństwa strony;
- `--no-dump` – wyłączenie zapisu logów JSONL (zalecane przy dużej liczbie gier, gdy nie potrzebujemy replayu);

- `--dump` – włączenie zapisu (domyślne); każda rozgrywka zapisuje jeden plik w katalogu `./logs` (np. `game_YYYYMMDD_HHMMSS_*.jsonl`).

Przykłady:

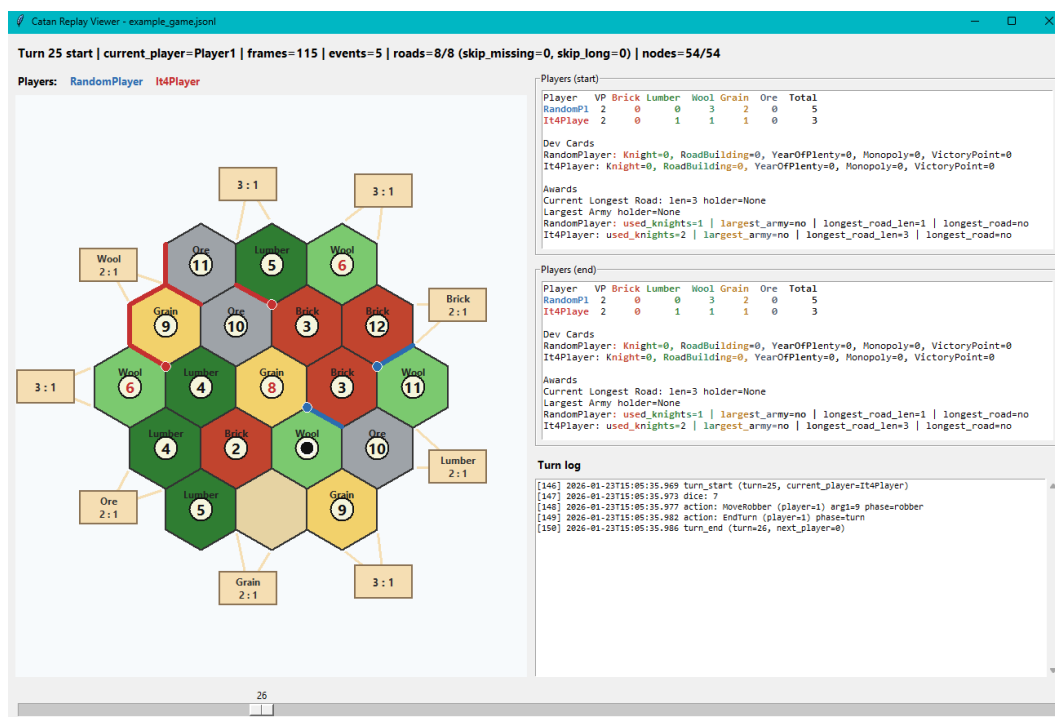
- jedna gra It4 vs It5 z zapisem logu: `run it4 it5`;
- 1000 gier Para vs Random bez logów: `run -n 1000 --no-dump para rp`;
- 500 gier z przełączaniem stron i zapisem: `run -n 500 --switch ab it5`.

Odtwarzanie i wizualizacja (replay viewer). Narzędzie `utils/replay_viewer.py` wczytuje plik JSONL wygenerowany przez Dumper i wizualizuje przebieg gry. Uruchomienie:

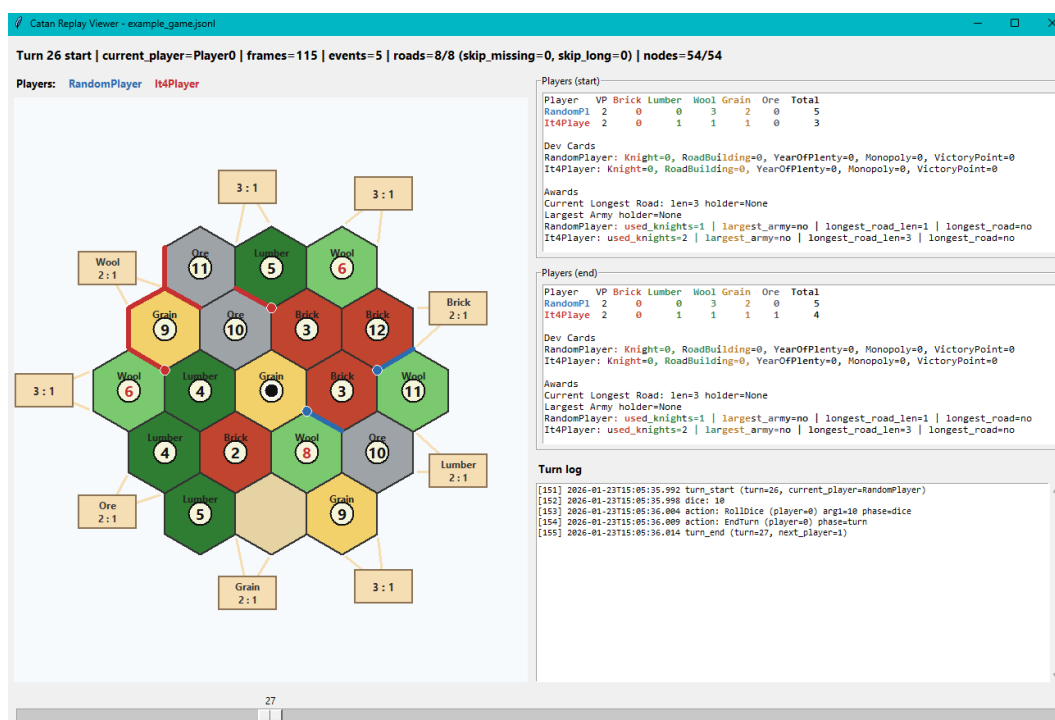
```
python utils/replay_viewer.py [sciezka/do/pliku.jsonl]
```

Jeśli nie podano ścieżki, otwiera się okno wyboru pliku. Opcja `-lr` (lub `--last-replay`) automatycznie wybiera najnowszy plik `*.jsonl` z katalogów `logs/` lub `build/logs/` i uruchamia odtwarzacz z włączonym auto-odtworzeniem.

W oknie odtwarzacza: suwak pozwala przewijać stany gry (początek, kolejne tury); klawisze strzałek zmieniają klatkę; spacja włącza lub zatrzymuje auto-odtworzenie. Wyświetlane są: plansza (hekсы z zasobami i numerami, rozbójnik, drogi, osady i miasta, porty), tabele graczy (punkty zwycięstwa, zasoby, karty rozwoju, premie) oraz log zdarzeń dla bieżącej tury. Dzięki temu można krok po kroku prześledzić rozgrywkę i zlokalizować błędy lub nieoczekiwane zachowania botów.



Rysunek 5.1: Przykład działania odtwarzacza: po lewej stronie stan planszy na początku tury 26 w której wypadła siódemka i gracz It4 przestawił rozbójnika na inny heks



Rysunek 5.2: Stan planszy na początku tury 27.

Rozdział 6.

Testowanie i weryfikacja poprawności

Poprawność silnika gry jest warunkiem koniecznym sensowności części badawczej: nawet subtelne błędy reguł mogłyby fałszować wyniki porównania botów. W projekcie zastosowano testy automatyczne oparte o GoogleTest/GMock [11], uruchamiane przez CTest.

6.1. Strategia testowania

Przyjęta strategia testowania obejmuje:

- testy jednostkowe dla komponentów niskiego poziomu (np. pakowanie i rozpakowywanie struktur),
- testy integracyjne obejmujące wykonanie sekwencji akcji na stanie gry,
- testy własnościowe w ograniczonym zakresie, w szczególności dla mechanizmu `applyAction()` / `undoLastAction()`.

W testach preferuje się deterministyczne scenariusze (kontrolowane stany wejściowe i sekwencje akcji), co umożliwia regresję i jednoznaczną interpretację niepowodzeń.

6.2. Testy jednostkowe i integracyjne

Zestaw testów obejmuje m.in.:

- poprawność generowania akcji legalnych w danym stanie (walidacja ograniczeń budowy, handlu i kart rozwoju),

- poprawność aktualizacji zasobów graczy i banku,
- zachowanie premii *Najdłuższa Droga* i *Największa Armia*,
- scenariusze obejmujące fazę początkową, pełne tury oraz warunki zakończenia gry.

Testy integracyjne wykorzystują rzeczywiste wywołania metod silnika (`applyAction`), dzięki czemu weryfikują zachowanie systemu w sposób zbliżony do uruchomień eksperymentalnych.

6.3. Walidacja zgodności z zasadami gry

Odrębnym celem testów jest walidacja zgodności implementacji z zasadami gry (w zakresie przyjętego wariantu 1 vs 1). W szczególności testuje się:

- zasadę dystansu dla osad i miast,
- warunki legalnej budowy (wymóg połączenia drogą, dostępność elementów),
- działanie rozbójnika i konsekwencje wyrzucenia 7,
- odwracalność sekwencji akcji: wykonanie akcji i jej cofnięcie przywraca stan do wartości wyjściowej.

W połączeniu z logowaniem do JSONL oraz odtwarzaczem (rozdział 5) testy tworzą spójny proces weryfikacji: testy wykrywają regresje automatycznie, a replay viewer umożliwia szybkie zrozumienie przyczyn błędów w kontekście całej rozgrywki.

Rozdział 7.

Projekt i implementacja botów

W rozdziale przedstawiono zaprojektowanych i zaimplementowanych graczy automatycznych (boty), które zostały wykorzystane do badań porównawczych w dalszej części pracy. Boty różnią się stopniem złożoności strategii decyzyjnej – od gracza w pełni losowego, pełniącego rolę punktu odniesienia, po boty heurystyczne rozwijane iteracyjnie poprzez stopniowe wzbogacanie funkcji oceny stanu gry. Dodatkowo zaprezentowano bota parametrycznego, którego strategia opiera się na tej samej strukturze heurystycznej, lecz sterowana jest zestawem konfigurowalnych parametrów, umożliwiającym łatwą modyfikację zachowania oraz eksperymenty z dostrajaniem strategii.

Architektura botów została zaprojektowana w oparciu o wzorzec projektowy **Strategy**. Silnik gry współpracuje z botami poprzez wspólny interfejs gracza, natomiast konkretne implementacje strategii decyzyjnej są enkapsulowane w klasach poszczególnych botów. Umożliwia to wymienne stosowanie różnych algorytmów podejmowania decyzji bez konieczności modyfikacji logiki silnika, a także ułatwia prowadzenie eksperymentów porównawczych pomiędzy strategiami [14].

7.1. Gracz losowy (baseline)

Najprostszym zaimplementowanym graczem automatycznym jest **gracz losowy**, który stanowi punkt odniesienia dla wszystkich pozostałych botów. Jego głównym celem nie jest osiąganie wysokich wyników, lecz dostarczenie **minimalnego poziomu kompetencji**, względem którego można mierzyć skuteczność bardziej zaawansowanych strategii.

7.1.1. Założenia projektowe

Gracz losowy działa według następujących zasad:

- w każdej fazie gry pobiera od silnika listę **wszystkich legalnych akcji** dostępnych w danym stanie,
- wybiera jedną z nich z **jednakowym prawdopodobieństwem**,
- nie analizuje przyszłych konsekwencji decyzji,
- nie wykorzystuje żadnej wiedzy domenowej o grze *Catan*.

Bot ten nie posiada pamięci długoterminowej ani mechanizmu uczenia – każda decyzja podejmowana jest niezależnie od poprzednich ruchów oraz aktualnej sytuacji strategicznej.

7.1.2. Rola w badaniach

Pomimo swojej prostoty, gracz losowy pełni istotną funkcję w pracy:

- umożliwia weryfikację poprawności działania silnika gry (czy gra dobiega do końca bez błędów),
- pozwala określić **dolną granicę skuteczności** strategii,
- stanowi punkt odniesienia przy ocenie, czy dana heurystyka faktycznie wnosi wartość decyzyjną.

Jeżeli bot heurystyczny nie osiąga statystycznie lepszych wyników niż gracz losowy, oznacza to, że zaprojektowana strategia jest nieskuteczna lub błędnie zaimplementowana.

7.1.3. Charakterystyka zachowania

W praktyce gracz losowy:

- często buduje struktury w nieoptymalnych lokalizacjach,
- nie planuje ciągłości sieci dróg,
- zużywa zasoby bez długoterminowego celu,
- podejmuje losowe decyzje dotyczące kart rozwoju i budowy.

Pomimo tego, dzięki losowości rzutów kośćmi, bot ten jest w stanie okazjonalnie wygrać pojedyncze rozgrywki, co dodatkowo podkreśla znaczenie przeprowadzania **dużej liczby symulacji** w analizie wyników.

7.1.4. Boty heurystyczne

Boty heurystyczne rozwijane były iteracyjnie: każda kolejna wersja rozszerza poprzednią o nowe elementy strategii. Dzięki temu można ocenić wpływ poszczególnych mechanizmów na skuteczność oraz obserwować, jak nawet proste heurystyki poprawiają decyzje względem losowego wyboru akcji.

it1 – priorytet punktów zwycięstwa

It1 wybiera akcje według **stałej hierarchii priorytetów**: najpierw budowa miasta, potem osady, drogi, karty rozwoju, na końcu handel i inne. W obrębie każdej kategorii wybór jest **losowy**. Bot nie handluje z bankiem, nie optymalizuje ustawień początkowych ani rozbójnika. Strategia jest prosta i krótkowzroczna — maksymalizuje natychmiastowy przyrost punktów, nie uwzględniając przyszłej produkcji ani pozycji na planszy.

it2 – handel z bankiem i cel zakupu

It2 rozszerza it1 o **aktywne wykorzystanie handlu z bankiem**. Wykrywa kontrolowane porty (2:1, 3:1, 4:1) i ocenia, **do którego celu zakupu jest najbliższej po uwzględnieniu możliwych wymian** — tzn. ile brakujących surowców pozostaje po optymalnym “wydaniu” nadwyżek. Cel wybierany jest nie na podstawie tego, co da się kupić od razu, lecz tego, do czego po handlu jest się najbliższej. Bot może odłożyć budowę drogi i najpierw handlować, jeśli po wymianie będzie mógł zbudować osadę lub miasto. Hierarchia: miasto i osada (natychmiast) → handel w kierunku wybranego celu → droga → karta rozwoju → koniec tury.

it3 – ustawienia początkowe i rozbójnik

It3 dodaje **świadomą fazę początkową** oraz **celowe ustawianie rozbójnika**. Ustawienia początkowe oceniane są według produkcji (z naciskiem na surowce ważne we wczesnej grze), różnorodności zasobów i dostępu do portów. Drugie ustawienie dobierane jest tak, aby **uzupełniać braki** po pierwszym — bot dąży do pokrycia wszystkich typów surowców. Przy ruchu rozbójnikiem bot wybiera heks, który **maksymalnie ogranicza produkcję przeciwnika przy minimalnym wpływie na własną**. W fazie głównej zachowuje logikę it2 (handel i cele zakupu).

it4 – karty rozwoju i deterministyczny wybór budowy

It4 wprowadza **inteligentne używanie kart rozwoju** oraz **deterministyczny wybór lokalizacji** zamiast losowego rozstrzygania remisów podczas budowy osad

i miast. Dla każdego typu karty (Rycerz, Monopol, Rozwój itd.) stosowana jest odrębna logika: Rycerze — blokada przeciwnika i premia Największa Armia, karty zasobowe — pod kątem odblokowania budowy miasta lub osady, karty VP — jako bezpośredni zysk punktowy. Uwzględniane jest **ryzyko przed rzutem** (7): unika się akcji, które mocno powiększają rękę przed rzutem. Budowa (miasto, osada, droga) wybierana jest deterministycznie na podstawie **potencjału produkcyjnego** i użyteczności sieci dróg; priorytety dostosowują się do fazy gry.

it5 – ocena pozycji i jednokrokowy lookahead

It5 uzupełnia it4 o **jednokrokową symulację** (*one-step lookahead*) i **zunifikowaną ocenę pozycji**. Zamiast sztywnych priorytetów bot dla każdej rozważanej akcji deterministycznej: stosuje ją do kopii stanu, liczy wartość funkcji oceny pozycji, cofa akcję — i wybiera akcję dającą **najwyższą ocenę**. Decyzja opiera się więc na **rzeczywistym wpływie na stan gry**, a nie na samym typie akcji. Najpierw rozważane są budowy miasta i osady (wybór najlepszej lokalizacji), potem drogi i handel; dodatkowo premiowane są ruchy, które **odblokowują** w następnym kroku budowę miasta lub osady.

Zakup karty rozwoju nie jest symulowany (losowość talii) i oceniany heurystycznie. Gdy najlepszą opcją jest zakończenie tury bez poprawy, bot **zawraca do logiki it4**. Przy wyrzuceniu 7 it5 **nie odrzuca losowo**: ustala najbliższy cel (miasto, osada, droga, karta), po czym odrzuca surowce najmniej potrzebne do tego celu, chroniąc zasoby krytyczne. It5 łączy deterministyczny wybór i sensowne priorytety z oceną konsekwencji ruchu przy niewielkim koszcie obliczeniowym.

7.2. Bot wykorzystujący algorytm alpha-beta

Kolejnym graczem automatycznym jest bot wykorzystujący algorytm przeszukiwania drzewa gry **alpha-beta**. Jego celem jest podejmowanie decyzji na podstawie analizy przyszłych stanów gry, z uwzględnieniem możliwych odpowiedzi przeciwnika. W przeciwieństwie do botów heurystycznych, które oceniają jedynie pojedynczy ruch, bot alpha-beta eksploruje sekwencje akcji, traktując rozgrywkę jako dwuosobową grę o sumie zerowej, w której:

- bot (gracz `selfId`) jest stroną maksymalizującą,
- przeciwnik jest stroną minimalizującą wartość funkcji oceny.

7.2.1. Reprezentacja drzewa i znaczenie głębokości

Drzewo decyzyjne budowane przez algorytm składa się z kolejnych stanów plan-szy, powstałych w wyniku zastosowania akcji gry. Głębokość przeszukiwania (`depth`)

w tej implementacji oznacza liczbę kolejnych **symulowanych akcji**, a nie liczbę pełnych tur. Ponieważ pojedyncza tura w grze składa się z wielu możliwych decyzji (np. handel, budowa, zakończenie tury), algorytm może zakończyć analizę w dowolnym momencie sekwencji ruchów.

7.2.2. Ograniczanie rozmiaru drzewa przeszukiwania

Ze względu na dużą liczbę potencjalnych akcji, bot stosuje mechanizmy redukujące rozgałęzienie drzewa:

1. **Filtrowanie akcji deterministycznych** – do przeszukiwania wybierane są głównie akcje o przewidywalnych skutkach, natomiast akcje silnie losowe są w większości pomijane.

2. **Selekcja najlepszych kandydatów** – akcje są wstępnie oceniane heurystycznie, sortowane i ograniczane do stałej liczby (14), przy czym zawsze gwarantowana jest obecność akcji **EndTurn**.

Dodatkowo stosowane jest porządkowanie ruchów (move ordering), co zwiększa skuteczność obcinania alpha-beta poprzez wcześniejsze rozpatrywanie najbardziej obiecujących decyzji.

7.2.3. Algorytm alpha-beta

Rdzeniem bota jest klasyczny algorytm minimax z obcinaniem alpha-beta, zaimplementowany z głębokością 3 akcji. Dla danego stanu gry:

- jeżeli osiągnięto maksymalną głębokość przeszukiwania ($\text{depth} = 0$), zwracana jest wartość funkcji oceny pozycji,
- w przeciwnym razie generowane są legalne akcje dla aktualnego gracza, które następnie podlegają filtrowaniu i sortowaniu.
- dla każdej akcji tworzona jest kopia stanu planszy i symulowane jest wykonanie akcji,
- jeśli akcja to **EndTurn**, automatycznie wywoływana jest symulacja oczekiwanych rzutów kością,
- jeśli ruch należy do bota (maximizing), wybierana jest akcja maksymalizująca ocenę, aktualizowane jest **alpha**, a przeszukiwanie przerywane jest gdy $\text{alpha} \geq \text{beta}$ (beta cutoff),
- jeśli ruch należy do przeciwnika (minimizing), wybierana jest akcja minimalizująca ocenę, aktualizowane jest **beta**, a przeszukiwanie przerywane jest gdy $\text{alpha} \geq \text{beta}$ (alpha cutoff).

Parametry **alpha** i **beta** reprezentują odpowiednio najlepszą wartość znaną dla gracza maksymalizującego oraz najlepszą wartość znaną dla gracza minimalizującego. Mechanizm przycinania pozwala na pominięcie gałęzi drzewa, które z pewnością nie będą wybrane, co znacząco redukuje liczbę rozważanych stanów.

7.2.4. Funkcja oceny pozycji i fazy gry

Ocena stanu gry realizowana jest przez funkcję heurystyczną `evaluate_position_stage()`, której struktura zależy od aktualnej fazy rozgrywki (wczesnej, środkowej lub późnej). Faza gry określana jest na podstawie maksymalnej liczby VP uzyskanych przez graczy oraz numeru tury.

We wczesnej fazie gry funkcja oceny kładzie większy nacisk na rozwój ekonomiczny, w szczególności na produkcję zasobów oraz potencjał dalszej ekspansji osadniczej. W fazie środkowej wagi poszczególnych składników są bardziej zrównoważone, natomiast w końcowej fazie gry dominującym czynnikiem staje się bezpośrednia realizacja punktów zwycięstwa, a znaczenie produkcji zasobów ulega względnemu zmniejszeniu.

Funkcja oceny uwzględnia następujące komponenty:

- **Punkty zwycięstwa** – kluczowy czynnik oceny, odzwierciedlający bezpośrednie zbliżanie się do warunku zakończenia gry.
- **Produkcję zasobów** – bieżący potencjał ekonomiczny gracza wynikający z rozmieszczenia osad i miast, porównywany z potencjałem przeciwnika.
- **Potencjał osadniczy** – ocenę możliwości dalszej ekspansji, wynikającą z aktualnej sieci dróg i dostępnych lokalizacji pod osady.
- **Deficyty zasobów** – stopień braków zasobów niezbędnych do realizacji kluczowych zakupów, takich jak miasto, osada czy karta rozwoju.
- **Ryzyko utraty kart** – kara za nadmierną liczbę kart zasobów na ręce, zwiększającą podatność na straty w przypadku wyrzucenia liczby 7.
- **Różnice w zasobach i kartach rozwoju** – porównanie aktualnej siły ekonomicznej i długoterminowych inwestycji obu graczy.
- **Najdłuższą drogę** – element strategiczny związany z kontrolą infrastruktury i dodatkowymi punktami zwycięstwa.
- **Oczekiwany zysk z rzutów kością** – heurystyczną ocenę potencjalnej przyszłej produkcji zasobów, wyznaczaną na podstawie rozkładu prawdopodobieństwa rzutów.
- **Możliwość natychmiastowej budowy** – dodatkowe premiowanie stanów, w których gracz może bezpośrednio zrealizować istotną akcję budowy.

Tak skonstruowana funkcja oceny łączy informacje krótkoterminowe (aktualne zasoby, możliwość budowy) z oceną długofalowego potencjału pozycji, umożliwiając algorytmowi alpha-beta podejmowanie decyzji lepiej dopasowanych do aktualnej fazy rozgrywki.

7.2.5. Symulacja rzutów kością w drzewie przeszukiwania

Aby uwzględnić wpływ losowości produkcji zasobów bez rozgałęziania drzewa na 36 możliwych wyników rzutu, przy każdej akcji **EndTurn** w drzewie przeszukiwania wywoływana jest funkcja **symulująca oczekiwaną produkcję** z jednego “przyszłego” rzutu kością. Zamiast losować konkretną sumę (2–12), bot oblicza **wartość oczekiwaną** produkcji: dla każdego gracza sumuje się, po wszystkich jego osadach i miastach, wkład każdego przyległego heksa. Dla heksa z liczbą o prawdopodobieństwie $pips/36$ (gdzie $pips$ to liczba sposobów wyrzucenia tej liczby) wkład to $pips$ (dla osady) lub $2 \cdot pips$ (dla miasta) — są to **liczniki ułamka $pips/36$ zasobu**. Heks zajęty przez rozbójnika jest pomijany (produkcja z niego 0). Suma tych liczników jest dodawana do **reszty z poprzedniej symulowanej tury** (tablica `remainderFP` dla każdego gracza): łączna wartość jest dzielona przez 36 — tylko całe karty (część całkowita z dzielenia) są dopisywane do ręki gracza w skopiowanym stanie, a reszta z dzielenia (0–35) jest zapisywana w `remainderFP` na następne wywołanie. Dzięki temu przy wielu kolejnych **EndTurn** w tej samej gałęzi drzewa ułamkowa “produkcja” się kumuluje i po kilku symulowanych turach gracz otrzymuje kolejne całe karty, bez gubienia ułamków. Stan przekazywany w głąb rekurencji alpha-beta zawiera zaktualizowane ręce obu graczy, a kolejne **EndTurn** w tej gałęzi dalej dokładają oczekiwaną produkcję i reszty są spójne między “turami”. W efekcie drzewo nie rozgałęzia się na wyniki kostki — zamiast tego **jedna** następna pozycja po **EndTurn** odzwierciedla średni przyrost zasobów z jednego rzutu, a wielokrotne **EndTurn** w gałęzi symulują wielokrotne rzuty w sposób przybliżony i deterministyczny.

1. **Wykrywanie zakończenia tury:** Gdy algorytm symuluje akcję **EndTurn** w drzewie decyzyjnym, automatycznie wywoływana jest funkcja `addExpectedResources()`, która modyfikuje kopię stanu planszy dla gracza, który właśnie zakończył turę.

2. **Obliczanie oczekiwanej produkcji:** Funkcja iteruje przez wszystkie węzły (osady i miasta) należące do danego gracza. Dla każdego węzła:

- Określa typ struktury (osada lub miasto), gdzie miasta mają mnożnik produkcji równy 2, a osady 1,
- Sprawdza wszystkie sąsiednie heksy produkcyjne (pomijając pustynie),
- Dla każdego heksa pobiera liczbę **pipsów** (odzwierciedlającą prawdopodobieństwo wystąpienia danej liczby na kostkach),

- Oblicza oczekiwaną produkcję dla danego typu zasobu jako $(\text{pips} \times \text{multiplier}) / 12$, gdzie dzielenie przez 12 normalizuje wartość do oczekiwanej produkcji na turę.

3. **Dodawanie zasobów do planszy:** Funkcja dodaje obliczone oczekiwane zasoby do aktualnych zasobów gracza w kopii stanu planszy.

Mechanizm ten wykorzystuje uproszczony model probabilistyczny: zamiast symulować wszystkie możliwe wyniki rzutów kością (co prowadziłoby do znacznego rozgałęzienia drzewa), algorytm oblicza **wartość oczekiwaną produkcji** na podstawie rozkładu prawdopodobieństwa rzutów.

Bez zastosowania tego podejścia algorytm oceniałby pozycje wyłącznie na podstawie aktualnych zasobów, pomijając fakt ich przyrostu w następnej turze. To mogłoby prowadzić do suboptymalnych decyzji, szczególnie w sytuacjach, gdy oczekiwana produkcja wpływa na dostępność dalszych opcji budowy.

7.2.6. Heurystyki uzupełniające i mechanizmy awaryjne

Po zakończeniu przeszukiwania drzewa bot stosuje dodatkowe heurystyki, m.in. ocenę zakupu karty rozwoju poza algorytmem alpha-beta, ze względu na jej losowy charakter. W sytuacjach, w których najlepszą akcją okazuje się **EndTurn**, a jej ocena jest gorsza niż aktualna pozycja, bot deleguje decyzję do prostszego gracza heurystycznego, co zapobiega nadmiernie pasywnemu stylowi gry.

7.3. Boty celujące w jedną strategię

7.3.1. Bot celujący w jeden zasób

Oprócz botów opisanych wcześniej zaimplementowano również dodatkowego gracza o wąsko wyspecjalizowanej strategii, nazwanego **OneResourcePlayer**. Jego założeniem jest maksymalizacja korzyści z jednego, wybranego surowca poprzez:

- wybór **priorytetowego surowca** na podstawie aktualnej konfiguracji planszy,
- zajęcia **portu 2:1** dla tego surowca już w ustawieniach początkowych,
- prowadzenie rozwoju infrastruktury (osady, miasta, drogi) w kierunku pól oraz portów związanych z tym surowcem,
- wykorzystanie nadprodukcji prioritytetowego zasobu do handlu z bankiem po korzystnym kursie 2:1.

Celem tego gracza jest sprawdzenie hipotezy: **czy silna specjalizacja w jeden surowiec i wczesne pozyskanie portu 2:1 może stanowić skuteczną**

strategię w wariancie 1 vs 1. Z tego względu bot ten należy traktować jako **bot eksperymentalny**.

7.3.2. Wybór priorytetowego surowca

Priorytetowy surowiec wybierany jest automatycznie na podstawie parametrów planszy. Dla każdego surowca obliczane są cechy opisujące jego „atrakcyjność”:

- suma pipsów ze wszystkich heksów danego surowca (ogólna dostępność produkcji),
- najlepszy pojedynczy węzeł (ile pipsów danego surowca może generować jedna osada),
- najlepszy węzeł połączony z portem 2:1 dla tego surowca (kluczowe dla strategii),
- liczba heksów z danym surowcem (różnorodność źródeł).

Na podstawie powyższych cech obliczana jest łączna ocena atrakcyjności każdego surowca. Preferowane są surowce zapewniające wysoką i stabilną produkcję, szczególnie w połączeniu z dostępem do portu 2:1. Dodatkowo uwzględniana jest ogólna dostępność surowca na planszy oraz jego znaczenie dla wczesnego rozwoju infrastruktury.

7.3.3. Ustawienie początkowe: wymuszenie portu 2:1

Najważniejszym elementem strategii OneResourcePlayer jest faza początkowa. Bot stara się tak dobrać pierwszą osadę, aby:

1. znajdowała się na węźle z portem 2:1 odpowiadającym wybranemu surowcowi, 2. miała możliwie wysoką produkcję tego surowca (wysokie pipsy), 3. zapewniała sensowną możliwość rozwoju (premiowany jest stopień węzła, tj. liczba dostępnych krawędzi do dalszej rozbudowy dróg).

Jeżeli wśród legalnych akcji nie ma możliwości uzyskania portu 2:1 dla najlepszego surowca (np. z uwagi na ograniczenia legalnych ustawień), bot podejmuje próbę znalezienia **jakiegokolwiek** portu 2:1 dostępnego w danym układzie i dostosowuje wybór priorytetowego surowca. Dopiero w ostateczności wybiera pierwszą legalną akcję.

Drugie ustawienie początkowe również preferuje surowiec priorytetowy, ale dodatkowo wprowadza heurystykę uzupełniania braków — premiowane są węzły dostarczające innych surowców, których bot nie miał w pierwszej osadzie, aby ograniczyć ryzyko całkowitego zablokowania rozwoju.

7.3.4. Logika tury: specjalizacja i konwersja zasobów

W normalnej fazie gry bot rozważa wyłącznie akcje deterministyczne i stosuje następującą kolejność priorytetów:

1. **Budowa miast i osad** – silnie premiowana, szczególnie gdy zwiększa produkcję priorytetowego surowca lub znajduje się na porcie 2:1.

2. **Handel z bankiem** – kluczowy element strategii. Bot **wydaje nadwyżki priorytetowego surowca** (wykorzystując port 2:1) w zamian za zasoby potrzebne do budowy. Transakcje są wysoko oceniane gdy odblokowują możliwość zakupu miasta lub osady.

3. **Budowa drogi** – oceniana przez funkcję potencjału, która bada węzły w zasięgu 1–2 krawędzi i premiuje pipsy priorytetowego surowca w możliwych lokalizacjach osad oraz możliwości dalszej rozbudowy, ignorując miejsca naruszające regułę odległości.

Strategia ta jest celowo **jednostronna**: bot często poświęca równowagę zasobów na rzecz maksymalizacji jednego kanału ekonomicznego (produkcja + port 2:1 → konwersja → budowa).

7.3.5. Odrzucanie kart i mechanizm awaryjny

Bot implementuje własną politykę zrzucania kart przy przekroczeniu limitu, wybierając docelowy zakup i silnie chroniąc zarówno priorytetowy surowiec, jak i zasoby krytyczne dla najbliższego celu.

7.3.6. Mechanizm awaryjny: powrót do bota zbalansowanego

W sytuacji, gdy żaden z ruchów nie daje wyraźnej korzyści w ramach strategii monosuwrowcowej (wszystkie deterministyczne akcje mają gorsze oceny niż *It5Player* w analogicznej sytuacji), bot nie wykonuje losowych działań. Zamiast tego stosowany jest **fallback do It5Player**, czyli najbardziej zbalansowanego bota heurystycznego. Zapobiega to „utknięciu” strategii w stanach, w których dążenie do jednego surowca przestaje być racjonalne (np. brak portu 2:1, zablokowanie kluczowych lokalizacji przez przeciwnika).

7.3.7. Bot celujący w karty rozwoju

DevPlayer stanowi drugą strategię eksperymentalną, opartą na hipotezie, że **priorytetowe skupienie się na zakupie kart rozwoju** oraz szybkie osiąganie związanych z nimi bonusów (takich jak premia *Największa Armia* oraz karty punktów zwycięstwa) może w określonych warunkach stanowić skuteczną alternatywę dla

klasycznej ekspansji terytorialnej.

7.3.8. Ustawienie początkowe i logika tury

W fazie ustawień początkowych bot preferuje lokalizacje zapewniające stabilną produkcję surowców niezbędnych do zakupu kart rozwoju, w szczególności rudę, zboże i wełnę, kluczowych dla zakupu kart rozwoju. Premiowana jest różnorodność zasobów oraz dostęp do portów ułatwiających ich wymianę (porty 3:1 i 2:1). Drugie ustawienie wybierane jest w sposób komplementarny, tak aby uzupełnić ewentualne braki w produkcji kluczowych surowców.

W fazie głównej strategia koncentruje się na zakupie kart rozwoju zawsze, gdy jest to możliwe. Jeżeli bezpośredni zakup nie jest dostępny, bot podejmuje działania przygotowawcze — w szczególności handel z bankiem lub ograniczoną rozbudowę infrastruktury — których celem jest umożliwienie zakupu karty w kolejnej turze. Odstępstwo od tej zasady występuje jedynie w sytuacjach, gdy dostępna jest bezpośrednia akcja prowadząca do zakończenia gry poprzez zdobycie brakujących punktów zwycięstwa.

W przypadku gdy strategia oparta na kartach rozwoju przestaje przynosić oczekiwane efekty (np. brak postępu punktowego lub ograniczone możliwości zakupu kart), bot przechodzi do bardziej zbalansowanej strategii heurystycznej, wykorzystując mechanizm awaryjny oparty na it5.

7.3.9. Polityka odrzucania i ograniczenia strategii

Przy konieczności odrzucenia kart w wyniku wyrzucenia liczby 7 bot w pierwszej kolejności chroni surowce kluczowe dla zakupu kart rozwoju, natomiast pozbywa się nadwyżek zasobów o mniejszym znaczeniu dla realizowanej strategii. Takie podejście ogranicza ryzyko utraty postępu w kierunku kolejnych zakupów.

Ze względu na silne uzależnienie od losowości talii kart rozwoju oraz rzutów kośćmi, strategia ta charakteryzuje się większą wariancją wyników w porównaniu do botów heurystycznych i algorytmu alpha-beta. W ramach pracy DevPlayer pełni rolę **strategii porównawczej**, umożliwiającej ocenę skuteczności podejścia opartego na rozwoju pośrednim i wysokiej nieprzewidywalności.

7.3.10. Bot celujący w najdłuższą drogę

RoadPlayer stanowi trzecią strategię eksperymentalną, której celem jest weryfikacja hipotezy, że **agresywna rozbudowa sieci dróg oraz zdobycie premii Najdłuższa Droga** (2 punkty zwycięstwa) może stanowić efektywną alternatywę dla klasycznego podejścia opartego na szybkim rozwoju osad i miast.

7.3.11. Ustawienie początkowe i logika tury

W fazie ustawień początkowych bot preferuje lokalizacje zapewniające wysoką i stabilną produkcję surowców niezbędnych do budowy dróg, w szczególności cegły i drewna. Premiowana jest również różnorodność zasobów, wysoki stopień węzłów (większa liczba możliwych kierunków ekspansji) oraz dostęp do portów ułatwiających wymianę surowców związanych z infrastrukturą drogową. Drugie ustawienie wybierane jest w sposób komplementarny, tak aby ograniczyć ryzyko niedoborów kluczowych zasobów.

W fazie głównej strategia konsekwentnie faworyzuje **budowę dróg** jako podstawową akcję rozwojową. Bot preferuje drogi, które wydłużają istniejącą sieć, zwiększają jej spójność oraz otwierają dostęp do kolejnych obszarów planszy. Jednocześnie uwzględniany jest potencjał węzłów dostępnych po rozbudowie, co pozwala unikać sytuacji, w których sieć dróg rozwija się kosztem całkowitej izolacji ekonomicznej.

Aby zachować minimalną równowagę strategiczną, RoadPlayer nie rezygnuje całkowicie z budowy osad i miast. Akcje te są podejmowane wtedy, gdy bezpośrednio prowadzą do zdobycia punktów zwycięstwa lub gdy umożliwiają dalszą, efektywną rozbudowę sieci dróg. Zakup kart rozwoju traktowany jest drugorzędnie i rozważany głównie w sytuacjach, gdy inne formy ekspansji są chwilowo niedostępne.

7.3.12. Polityka odrzucania i ograniczenia strategii

Przy konieczności odrzucania kart w wyniku wyrzucenia liczby 7 bot w pierwszej kolejności chroni surowce bezpośrednio związane z budową dróg, natomiast usuwa nadwyżki zasobów o mniejszym znaczeniu dla realizowanej strategii. W sytuacjach, w których żadna dostępna akcja nie prowadzi do poprawy pozycji, stosowany jest mechanizm awaryjny oparty na bardziej zbalansowanej strategii heurystycznej.

Strategia RoadPlayer jest **wrażliwa na ograniczenia przestrzenne planszy**. Skuteczne blokowanie kluczowych węzłów przez przeciwnika lub przerwanie ciągłości sieci znacząco obniża jej skuteczność. Ponadto jednostronna koncentracja na infrastrukturze drogowej może prowadzić do utraty tempa zdobywania punktów zwycięstwa, szczególnie w starciu z botami preferującymi rozwój ekonomiczny i budowę miast. Wyniki uzyskane przez RoadPlayer potwierdzają, że silna specjalizacja w jednym aspekcie gry nie gwarantuje przewagi nad strategiami zbalansowanymi.

7.4. Boty oparte na algorytmie genetycznym

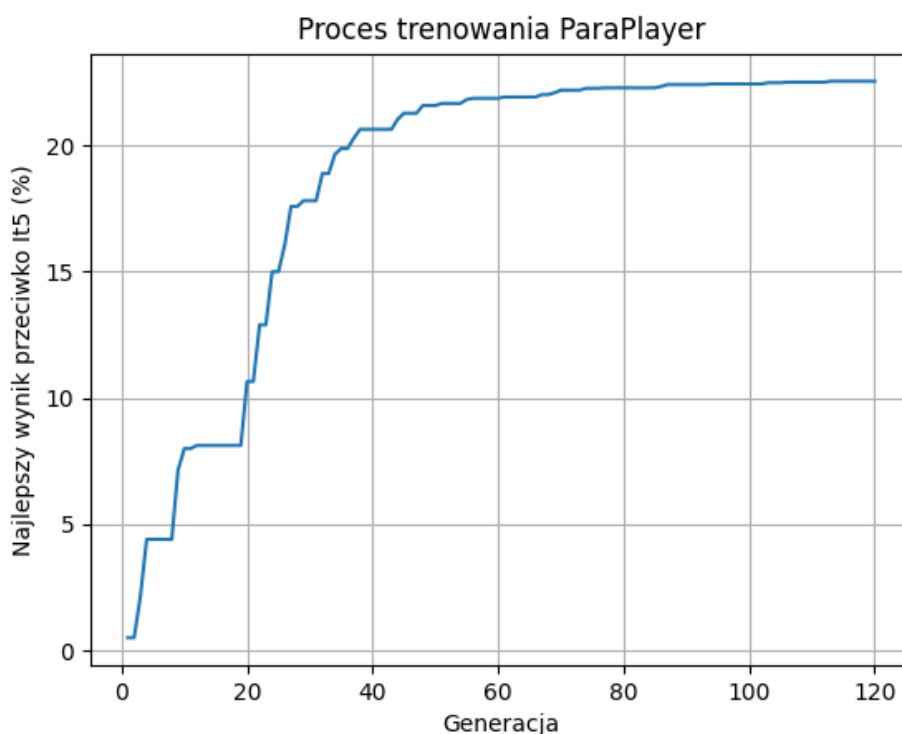
ParaPlayer

W ramach projektu zaimplementowano klasę gracza parametryzowanego (**ParaPlayer**), w której funkcja oceny i priorytety decyzyjne są sterowane zestawem wag wczytywa-

nych z pliku konfiguracyjnego (`players/paraPlayer.cfg`). Takie podejście umożliwia oddzielenie logiki wyboru akcji od doboru wartości parametrów.

Bot wykorzystuje liniową funkcję oceny pozycji (VP, produkcja, deficyty surowców, ręka, najdłuższa droga, karty rozwoju, wskaźniki “można zbudować”), premie i kary za typ akcji oraz osobną heurystykę rozbójnika i ustawień początkowych. Wybór akcji polega na ocenie każdej legalnej akcji (symulacja jednego ruchu), z opcjonalnym *lookahead* o jeden krok dla najlepszych M akcji. Przy odrzucaniu kart bot wybiera cel (miasto, osada, droga, karta) i odrzuca surowce o najniższej wadze dla tego celu. Plik konfiguracyjny zawiera dziesiątki parametrów (`policy`, `eval`, `prod`, `action`, `robber`, `init`, `discard`); wartości domyślne są zbliżone do It5 i nadpisywane przez trening.

Skrypt `utils/train_para_player.py` realizuje ewolucję: **kurikulum** przeciwników (`rp`, `it1`, ..., `it5`) z testem bramkowym, w każdej generacji **elita** plus mutanty (mutacja Gaussa $N(0, \sigma) \cdot (|v| + 1)$), `fitness` = procent zwycięstw w grach (`run` z `CATAN PARA_CFG`), opcjonalna weryfikacja nowego najlepszego i zapis do `paraPlayer.cfg`, łagodna zmiana σ przy poprawie i przy braku poprawy.



Rysunek 7.1: Proces trenowania ParaPlayer

Rysunek przedstawia przebieg treningu przeciwko It5 (oś odciętych: generacja 0–120, oś rzędnych: najlepszy % zwycięstw). W pierwszych generacjach wynik rośnie do ok. 4–5%, potem występuje skok (ok. 18–25) do ok. 17–20% i wzrost do ok. 22% w

okolicach generacji 35–45; po ok. 45. generacji krzywa wypłaszcza się na poziomie ok. 22–22,5%. Ilustruje to typową zbieżność ewolucyjną: szybka poprawa, potem plateau, gdy dalsze mutacje rzadko dają przewagę nad silnym przeciwnikiem. Otrzymany poziom ok. 22,5% vs It5 stanowi osiągnięty w eksperymencie szczyt skuteczności ParaPlayer.

W praktyce podejście to pełni dwie role: (1) automatyczne dostrajanie wag heurystyk oraz (2) przykład optymalizacji strategii na podstawie wyników rozgrywek, kompatybilnej z deterministycznym silnikiem symulacyjnym.

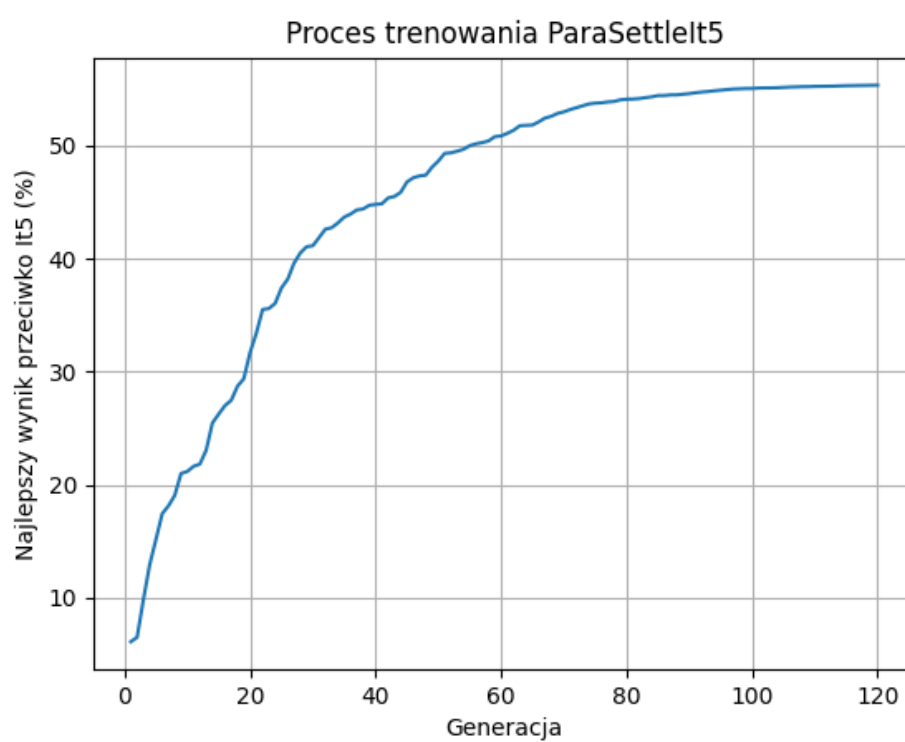
ParaSettleIt5Player

ParaSettleIt5Player jest hybrydą: dziedziczy po It5Player i korzysta z logiki It5 we wszystkich fazach gry **z wyjątkiem ustawień początkowych**. Tylko wybór pierwszej i drugiej osady oraz drogi jest parametryzowany i sterowany plikiem konfiguracyjnym (`players/paraSetit5Player.cfg`, zmienna `CATAN_PARA_SETIT5_CFG` przy treningu). Dzięki temu trenowana jest wyłącznie heurystyka ustawień początkowych przy stałej, sprawdzonej strategii It5 w dalszej części rozgrywki.

Plik konfiguracyjny zawiera mniejszy zestaw parametrów niż ParaPlayer: **policy** (temperatura, ε , top-k), **prod** (wagi surowców i portów w ocenie produkcji węzła) oraz **init** (skala produkcji, premie za różnorodność, porty i komplementarność drugiej osady, kara duplikatów, premia za stopień węzła przy drodze). Ocena węzła przy ustawieniu jest taka jak w ParaPlayer (produkcja przeskalowana + premie/kary), wybór akcji placementu — ε -zachłanny lub softmax/top-k.

Skrypt `utils/train_para_setit5_player.py` realizuje ewolucję parametrów w tym samym schemacie co trening ParaPlayer (elita + mutacje Gaussa, `fitness = % zwycięstw w grach` z `--switch`, opcjonalne successive halving i weryfikacja), przy kurikulum złożonym np. z przeciwnika It5.

Rysunek przedstawia przebieg treningu ParaSettleIt5 przeciwko It5 (oś odciętych: generacja 0–120, oś rzędnych: najlepszy % zwycięstw). Wynik startuje na ok. 6–7% i w pierwszych 40 generacjach rośnie szybko (ok. 18–20% w gen. 10, ok. 28–30% w gen. 20, ok. 45% w gen. 40). Potem tempo spada; próg 50% zostaje przekroczony w okolicach generacji 70–75, a ok. 90. generacji krzywa wypłaszcza się na poziomie ok. 54–55%. Oznacza to, że wytrenowany ParaSettleIt5 osiąga **ponad 50% zwycięstw** przeciwko It5 i staje się od niego skuteczniejszy. Mniejsza przestrzeń parametrów (tylko ustawienia początkowe) ułatwia zbieżność ewolucji w porównaniu z pełnym ParaPlayer, który w analogicznym eksperymencie ustabilizował się na ok. 22,5% vs It5.



Rysunek 7.2: Proces trenowania ParaSettleIt5

Rozdział 8.

Eksperymenty i analiza wyników

8.1. Metodologia porównania botów

Wszystkie eksperymenty zostały przeprowadzone zgodnie z następującym protokołem:

- **Liczba rozgrywek:** Każda para botów rozegrała **1000 rozgrywek**, co zapewnia statystyczną istotność wyników. Jeżeli w trakcie rozgrywki żaden z graczy nie osiągnął warunku zwycięstwa po ustalonej maksymalnej liczbie tur, partia była uznawana za nierozegraną.
- **Eliminacja efektu miejsca:** W celu wyeliminowania wpływu kolejności ustawień początkowych (gracz rozpoczynający posiada niewielką przewagę) włączono **naprzemienne ustawienie stron** (`--switch`). W każdej parze rozgrywek gracze zamieniają się miejscami, co zapewnia sprawiedliwe porównanie niezależne od pozycji startowej.
- **Losowość planszy:** Każda rozgrzywka wykorzystuje losowo wygenerowaną planszę zgodnie z zasadami gry *Catan*, co eliminuje możliwość optymalizacji strategii pod konkretną konfigurację terenu.

8.1.1. Metryki oceny skuteczności

W celu kompleksowej oceny skuteczności botów wprowadzono następujące metryki:

Metryki podstawowe

- **Procent wygranych rozgrywek:** Win Rate
- **Średnia liczba tur do końca gry:** Avg Turns

- Średnia liczba punktów przeciwnika przy przegranej: LossVP

Metryki dodatkowe

- Procent nierozegranych rozgrywek: NW (No Winner)
- Częstość zdobycia premii Najdłuższa Droga i Największa Armia: LR% (Longest Road) i LA% (Largest Army)
- Średnia liczba zakupionych kart rozwoju: Avg DevCards
- Średnia produkcja zasobów: ProdScore. Metryka potencjału produkcyjnego gracza na turę, wyznaczana ze stanu planszy po zakończeniu rozgrywki, w skali ważonej. Dla każdego pola zasobowego przy osadzie lub mieście: iloczyn liczby sposobów wyrzucenia danej sumy oczek (pips, z 36) i wagi zasobu (Ruda: 14, Zboże: 13, Cegła: 12, Drewno: 12, Wełna: 11). Miasta liczą się $2\times$, osady $1\times$. Porty dodają $+35$ (3:1) lub $+70$ (2:1).

8.2. Scenariusze testowe

Scenariusz 1: Porównanie z graczem losowym

W pierwszym scenariuszu **wszystkie boty** (kolejne iteracje, wersje parametryczne i strategie wyspecjalizowane) porównano z graczem losowym pełniącym rolę punktu odniesienia (baseline). Takie zestawienie pozwala bezpośrednio ocenić wpływ wzbogacania heurystyk na skuteczność, tempo rozgrywki oraz zdolność botów do deterministycznego domykania partii.

Podsumowanie danych

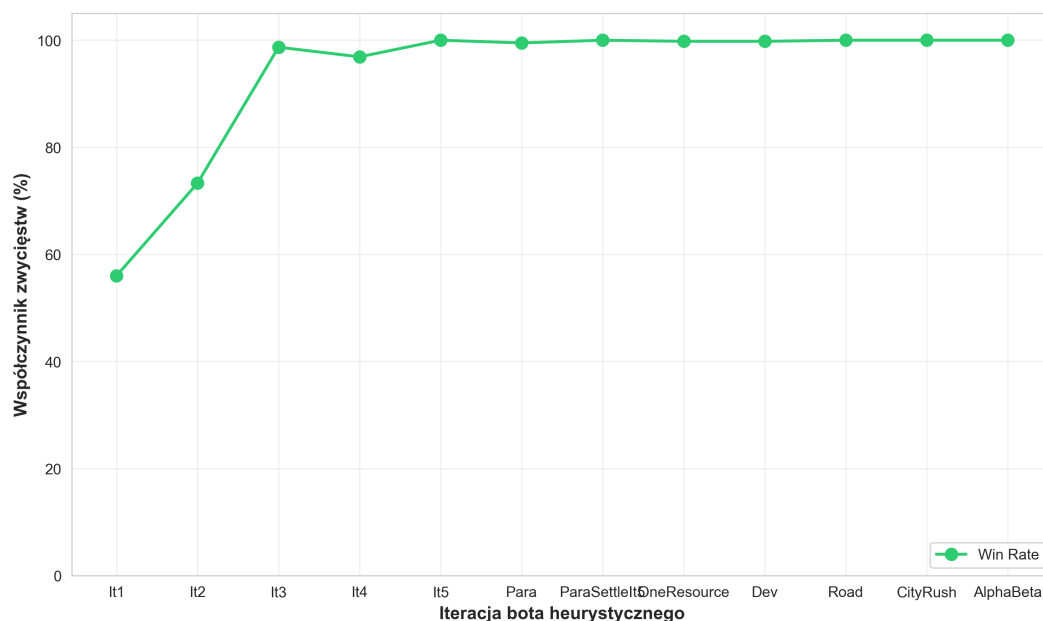
Tabela 8.1: Porównanie wyników w parach botów

Opponent	Win Rate	NW%	Avg Turns	LossVP (Random)	LossVP (Bot)	LR% (Bot)	LA% (Bot)	Avg DevCards	ProdScore
It1	56.0%	7.6%	448.5	6.81	7.51	51.2%	58.9%	3.0	730.4
It2	73.3%	3.6%	381.0	6.23	8.48	64.4%	62.0%	3.0	864.3
It3	98.7%	0.3%	180.3	3.98	11.50	83.7%	91.9%	3.2	1003.2
It4	96.9%	1.6%	192.6	3.64	9.80	94.6%	86.7%	3.0	1190.3
It5	100.0%	0.0%	115.3	3.21	–	71.5%	98.5%	3.7	1096.4
Para	99.5%	0.0%	125.1	3.34	4.80	90.2%	25.9%	9.9	1267.8
ParaSettleIt5	100.0%	0.0%	111.0	3.21	–	68.3%	98.8%	3.7	1111.5
OneResourcePlayer	99.8%	0.0%	159.9	3.79	11.50	82.0%	92.4%	2.9	1117.0
DevPlayer	99.8%	0.1%	166.1	4.00	12.00	52.8%	99.7%	4.6	1064.8
RoadPlayer	100.0%	0.0%	132.7	3.22	–	94.5%	98.6%	3.6	1051.0
CityRushPlayer	100.0%	0.0%	116.1	3.23	–	72.7%	97.7%	3.5	1076.1
AlphaBeta	100.0%	0.0%	108.7	2.97	–	96.2%	91.8%	2.5	1168.2

Kluczowe wnioski

Nieliniowy charakter progresji jakości

Wzrost skuteczności botów heurystycznych nie ma charakteru liniowego. Iteracje It1 i It2 prowadzą do stopniowej poprawy współczynnika zwycięstw (56% → 73%), natomiast It3 powoduje jakościowy skok skuteczności do 98.7%. Przekroczenie tego progu kompetencyjnego wyniku z wprowadzenia strategii optymalnych ustawień początkowych, zapewniających lepsze pozycje startowe, oraz aktywnego wykorzystania rozbójnika do blokowania produkcji przeciwnika. Od tego momentu bot przejmuje kontrolę nad przebiegiem rozgrywki, a kolejne iteracje stabilizują tę dominację, osiągając 96–100% zwycięstw.



Rysunek 8.1: Skuteczność kolejnych botów przeciwko graczowi losowemu.

Zależność tempa od jakości strategii

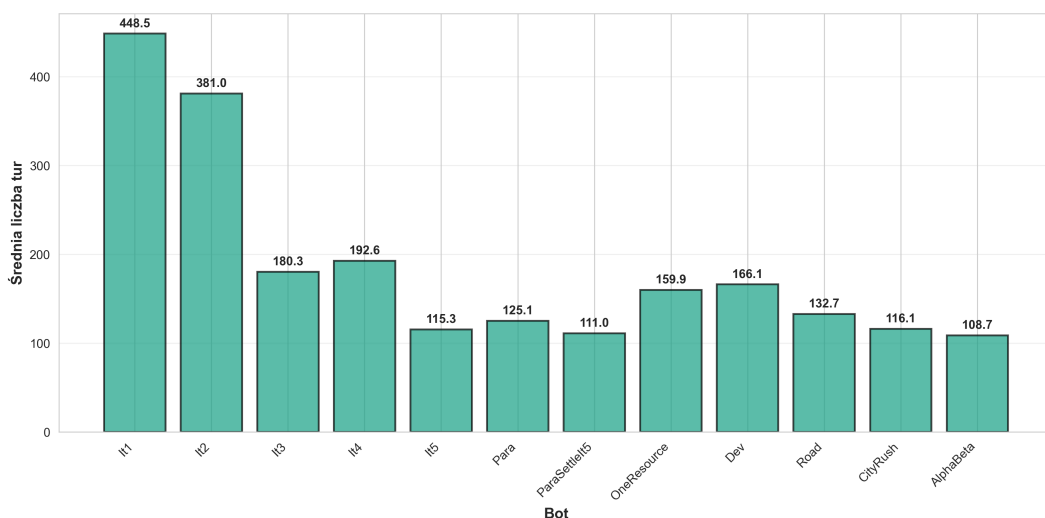
Analiza średniej liczby tur pokazuje wyraźną korelację między jakością strategii a tempem rozgrywki. Wczesne iteracje botów (It1, It2) charakteryzują się bardzo długimi rozgrywkami — odpowiednio 448.5 i 381.0 tur. Jest to konsekwencja niskiej skuteczności tych botów oraz braku zdolności do deterministycznego domykania partii, co prowadzi do przedłużających się rozgrywek, w których losowy przeciwnik ma realne szanse na zwycięstwo.

Przełom następuje wraz z wprowadzeniem It3, gdzie średnia liczba tur spada dramatycznie do 180.3, co stanowi redukcję o ponad 50% w stosunku do It2. Ta zmiana jest bezpośrednio związana z jakościowym skokiem skuteczności — bot przejmuje kontrolę nad rozgrywką i efektywnie kończy partie, nie pozwalając przeciwnikowi na przedłużanie rozgrywki.

Kolejne iteracje (It4, It5) oraz zaawansowane boty (Para, ParaSettleIt5, AlphaBeta) utrzymują szybkie tempo rozgrywki, osiągając średnią liczbę tur w zakresie

108–193. Szczególnie wyróżnia się AlphaBetaPlayer z najkrótszą średnią liczbą tur (108.7), co potwierdza, że algorytmy przeszukiwania drzewa gry są w stanie najszybciej identyfikować optymalne ścieżki prowadzące do zwycięstwa.

Warto zauważyć, że niektóre strategie wyspecjalizowane (OneResourcePlayer: 159.9 tur, DevPlayer: 166.1 tur) wykazują nieco wolniejsze tempo niż zaawansowane boty heurystyczne. Jest to związane z ich jednostronnym podejściem strategicznym, które może prowadzić do dłuższych rozgrywek, gdy specjalizacja nie jest wystarczająco efektywna.



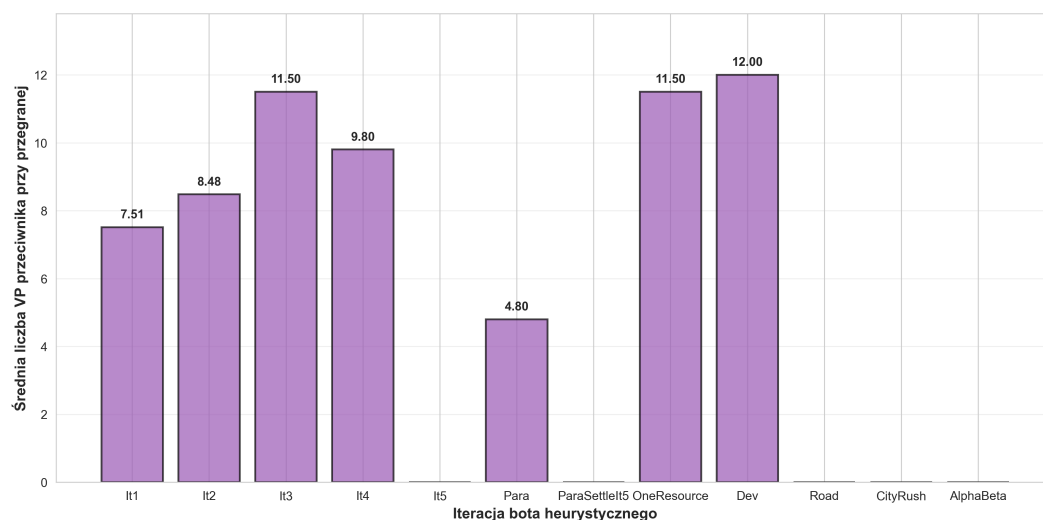
Rysunek 8.2: Zależność tempa rozgrywki od jakości strategii

Metryka LossVP

Metryka LossVP (Bot) mierzy średnią liczbę punktów zwycięstwa przeciwnika w momencie, gdy bot przegrywa rozgrywkę. Wyższa wartość tej metryki oznacza, że bot przegrywa w późniejszych fazach gry, gdy przeciwnik zdążył już zgromadzić znaczącą liczbę punktów, co może wskazywać na bardziej wyrównaną rozgrywkę.

Analiza wyników pokazuje interesujący wzorzec: wczesne iteracje (It1, It2) przegrywają przy stosunkowo niskich wartościach VP przeciwnika (7.51 i 8.48), co sugeruje, że przegrywają one w różnych fazach rozgrywki, często wcześnie. Wraz z poprawą jakości botów, wartości LossVP (Bot) rosną — It3 osiąga 11.50, a It4 9.80. To wskazuje, że bardziej zaawansowane boty przegrywają rzadziej, ale gdy już to robią, przeciwnik zdążył osiągnąć wyższy poziom rozwoju.

Szczególnie interesujące są wyniki dla strategii wyspecjalizowanych: DevPlayer osiąga najwyższą wartość LossVP (Bot) wynoszącą 12.00, podczas gdy OneResourcePlayer również osiąga 11.50. To sugeruje, że gdy te strategie przegrywają, dzieje się to w późniejszych fazach rozgrywki, gdy przeciwnik zdążył już rozwinąć swoją infrastrukturę.



Rysunek 8.3: Średnia liczba VP przeciwnika przy przegranej

Metryki dodatkowe

Analiza dodatkowych metryk pozwala na głębsze zrozumienie charakterystyki strategii poszczególnych botów oraz mechanizmów ich działania.

Premie strategiczne: Współczynniki zdobycia premii *Najdłuższa Droga* i *Największa Armia* pokazują wyraźne różnice w podejściu strategicznym. Boty heurystyczne (It3-It5) osiągają wysokie wartości dla obu premii (71-95% dla Najdłuższej Drogi, 87-99% dla Największej Armii), co wskazuje na zbalansowane podejście. ParaPlayer wyróżnia się bardzo wysokim współczynnikiem Najdłuższej Drogi (90.2%), ale niskim dla Największej Armii (25.9%), co jest związane z jego strategią ekspansji terytorialnej kosztem kart rozwoju. DevPlayer pokazuje odwrotny wzorec - dominuje w Największej Armii (99.7%), ale osiąga jedynie 52.8% dla Najdłuższej Drogi, co odzwierciedla jego specjalizację w strategii kartowej.

Karty rozwoju: Średnia liczba zakupionych kart rozwoju waha się od 2.5 (AlphaBeta) do 9.9 (ParaPlayer). ParaPlayer wyróżnia się zdecydowanie najwyższą wartością, co jest konsekwencją jego strategii opartej na eksploracji różnych opcji decyzyjnych. Większość botów heurystycznych utrzymuje wartość w zakresie 2.9-3.7 kart na rozgrywkę, podczas gdy DevPlayer, zgodnie ze swoją specjalizacją, osiąga 4.6 kart.

Produkcja zasobów: Metryka ProdScore pokazuje wyraźną progresję wraz z poprawą jakości botów. It1 osiąga 730.4, podczas gdy It4 osiąga najwyższą wartość wśród botów heurystycznych (1190.3). ParaPlayer osiąga najwyższą wartość w całym zestawie (1267.8), co potwierdza skuteczność jego zbalansowanego podejścia strategicznego. AlphaBetaPlayer osiąga 1168.2, co jest wartością wyższą niż większość botów heurystycznych, co wskazuje na efektywne wykorzystanie zasobów przez

algorytm przeszukiwania drzewa gry.

Wszystkie te metryki łącznie pokazują, że poprawa jakości botów przekłada się nie tylko na wyższy współczynnik zwycięstw, ale również na bardziej efektywne wykorzystanie mechanizmów strategicznych dostępnych w grze, co prowadzi do szybszych i bardziej deterministycznych rozgrywek.

8.2.1. Scenariusz 2: Analiza iteracyjnego rozwoju heurystyk

W scenariuszu 2 zestawiono iteracje botów heurystycznych w układzie „każdy z każdym” w celu weryfikacji, czy kolejne modyfikacje heurystyk (opisane w rozdziale 7.2) prowadzą do poprawy wyników również w bezpośrednim starciu z innymi wersjami, a nie wyłącznie przeciwko RandomPlayer. Każda para odpowiada testowi wpływu konkretnego rozszerzenia:

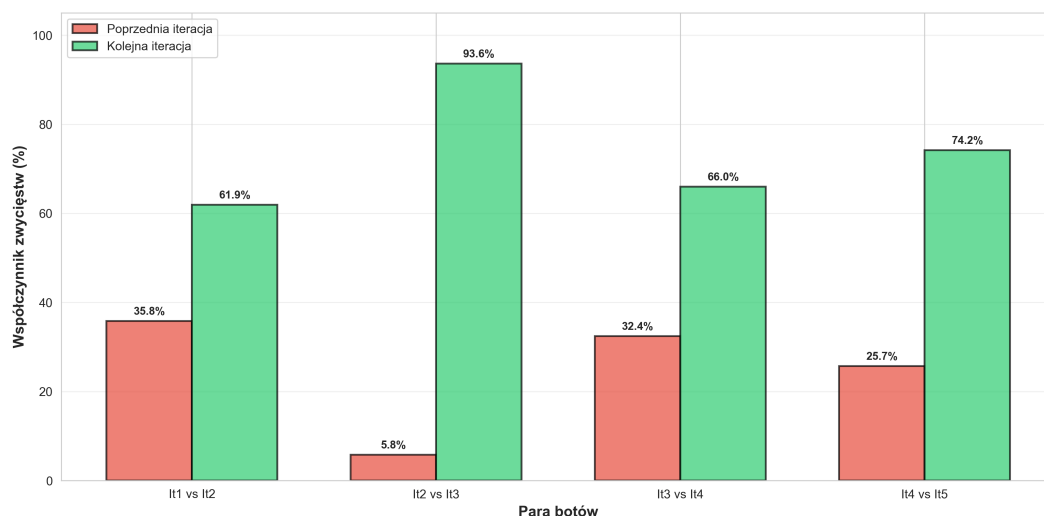
- **It1 vs It2** — wpływ wprowadzenia **handlu z bankiem** i **celu zakupu** (It2: handel w kierunku osady/miasta, wykrywanie portów). It2 wygrywa 61.9%, co potwierdza, że sama optymalizacja handlu znacząco poprawia wynik względem sztywnej hierarchii It1.
- **It2 vs It3** — wpływ **ustawień początkowych** (produkcja, różnorodność, porty) oraz **celowego ustawiania rozbójnika** (maksymalizacja szkody dla przeciwnika). It3 wygrywa 93.6%; ten największy skok jakościowy pokazuje, że dobra faza początkowa i kontrola rozbójnika są kluczowe.
- **It3 vs It4** — wpływ **kart rozwoju** (logika Rycerz/Monopol/zasobowe) oraz **deterministycznego wyboru budowy** (potencjał produkcyjny, unikanie ryzyka przed rzutem 7). It4 wygrywa 66.0%.
- **It4 vs It5** — wpływ **jednokrokowej symulacji** (lookahead) i **zunifikowanej oceny pozycji** zamiast sztywnych priorytetów oraz **deterministycznego odrzucania** przy siódemce. It5 wygrywa 74.2%.
- **It1 vs It5** — łączny efekt wszystkich warstw heurystyk: It5 osiąga 99.9% zwycięstw, co ilustruje przewagę oceny konsekwencji ruchu i pełnego zestawu mechanizmów nad prostą hierarchią It1.

Podsumowanie danych

Tabela 8.2: Porównanie wyników w parach botów

Pair	Win% A	Win% B	Avg Turns	LossVP A	LossVP B	LR% A	LR% B	LA% A	LA% B	DevCards A	DevCards B	ProdScore A	ProdScore B
It1 vs It2	35.8%	61.9%	339.8	6.24	7.96	37.4%	62.6%	45.8%	52.3%	2.3	2.6	589.1	832.1
It2 vs It3	5.8%	93.6%	181.9	5.06	11.28	31.4%	68.5%	5.8%	91.1%	0.9	3.2	400.5	1017.3
It3 vs It4	32.4%	66.0%	184.4	9.11	9.89	24.6%	75.4%	52.5%	46.3%	2.5	2.5	817.7	1127.4
It4 vs It5	25.7%	74.2%	134.2	9.54	11.04	82.0%	17.8%	11.4%	88.6%	1.7	3.4	958.1	1070.6
It1 vs It5	0.1%	99.9%	114.9	3.58	14.00	14.0%	73.5%	1.1%	98.2%	0.6	3.7	266.5	1096.8

Kluczowe wnioski

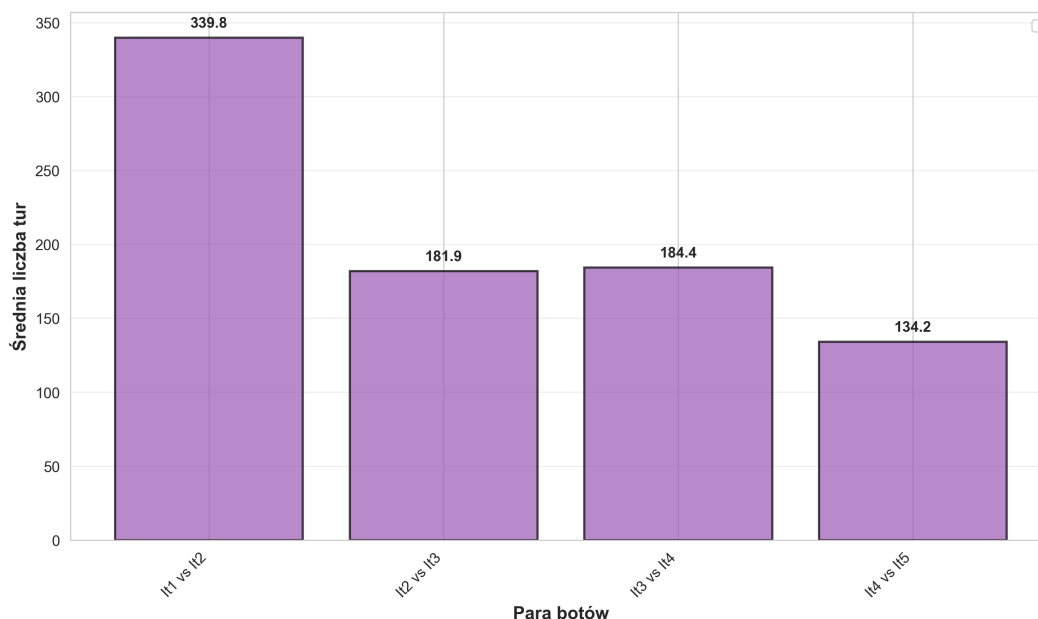


Rysunek 8.4: Porównanie wyników par botów w scenariuszu 2 (Win Rate).

Wyniki wskazują na **monotoniczną poprawę** w układzie iteracyjnym: w każdej z rozpatrywanych par nowsza iteracja uzyskuje wyższy współczynnik zwycięstw niż iteracja wcześniejsza. Interpretacja w świetle heurystyk z rozdziału 7.2:

- **It1 → It2:** Wprowadzenie handlu z bankiem i celu zakupu daje ok. 26 punktów procentowych przewagi (61.9% vs 35.8% dla It2). Średnia liczba tur spada z ok. 340 do ok. 182 w kolejnej parze, co potwierdza szybsze domykanie partii przez lepszą alokację zasobów.
- **It2 → It3:** Ustawienia początkowe i rozbójnik powodują **jakościowy skok** — It3 wygrywa 93.6% z It2. Spadek średniej liczby tur (ok. 182) oraz wyższy ProdScore It3 (1017.3 vs 400.5) pokazują, że lepsza pozycja startowa i blokowanie produkcji przeciwnika dają więcej zasobów.
- **It3 → It4:** Karty rozwoju i deterministyczny wybór budowy dają It4 wyraźną przewagę (66.0%), przy podobnym tempie (ok. 184 tury). Wzrost wykorzystania premii (np. Najdłuższa Droga 75.4% dla It4) odzwierciedla lepsze wykorzystanie kart i lokalizacji.
- **It4 → It5:** Jednokrokowy lookahead i zunifikowana ocena pozycji pozwalają It5 wygrywać 74.2% z It4 przy krótszych rozgrywkach (ok. 134 tury). It5 lepiej wykorzystuje Największą Armię (88.6% vs 11.4%), co wskazuje na skuteczniejszą strategię kartową i ocenę konsekwencji ruchu.

Najciekawsze momenty z wykresów premii. Na wykresie **Największa Armia (LA%)** wyróżniają się dwa skoki: przejście It2 → It3 (It2 tylko 5,8%, It3 aż

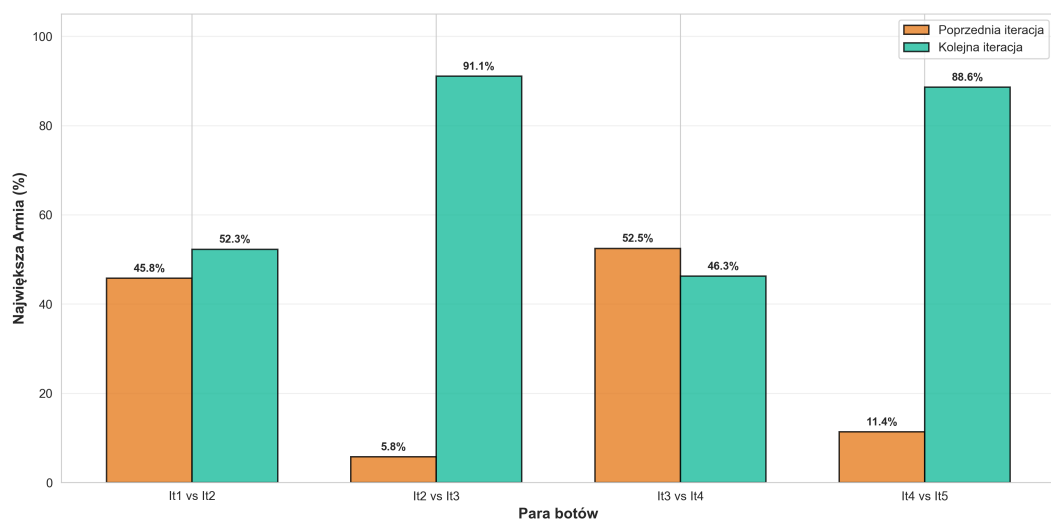


Rysunek 8.5: Porównanie średniej liczby tur (Avg Turns) dla wybranych par botów w scenariuszu 2.

91,1%) oraz It4 → It5 (It4 11,4%, It5 88,6%). Potwierdza to, że poprawki strategii dołączone do It3 i It5 silnie sprzyjają zdobywaniu premii przez karty rozwoju. W parze It3 vs It4 It4 ma nieco niższy LA% (46,3%) niż It3 (52,5%) — It4 bardziej koncentruje się na deterministycznym wyborze budowy i Najdłuższej Drodze niż na rycerzach. Na wykresie **Najdłuższa Droga (LR%)** widać stopniowy wzrost od It1 do It4 (It4 osiąga 82,0% w parze z It5), a następnie wyraźne odwrócenie w parze It4 vs It5: It5 ma tylko 17,8% LR%, podczas gdy It4 82,0%. It5 priorytetyzuje karty rozwoju i Największą Armię (88,6% LA%), więc w tej parze premia drogowa przechodzi na It4; mimo to It5 wygrywa 74,2% rozgrywek, co ilustruje, że wybór strategii (droga vs armia) zależy od heurystyk, a zwycięstwo nie wymaga przewagi we wszystkich metrykach.

Scenariusz 3: AlphaBetaPlayer

Scenariusz 3 weryfikuje skuteczność AlphaBetaPlayer — algorytmu wykorzystującego przeszukiwanie drzewa gry z przycinaniem alfa-beta — przeciwko różnym poziomom przeciwników. AlphaBetaPlayer bazuje na It5Player, ale wzbogaca go o przeszukiwanie na głębokości 3 z zaawansowaną funkcją oceny pozycji, która adaptuje się do fazy gry (wczesna/średnia/późna). Algorytm wykorzystuje filtrowanie akcji (maksymalnie 14 najlepszych akcji według heurystyki) oraz symuluje oczekiwane rzuty kostką przy kończeniu tury.



Rysunek 8.6: Największa Armia (LA%) w parach iteracji.

Podsumowanie danych

Tabela 8.3: Wyniki AlphaBetaPlayer vs przeciwnicy (Win Rate, tempo, premie)

Opponent	Win Rate (AB)	Avg Turns	LossVP (AB)	LossVP (Opp)	LR% (AB)	LR% (Opp)	LA% (AB)	LA% (Opp)
It1	99.9%	105.0	10.00	3.38	96.3%	2.7%	92.4%	3.9%
It2	99.5%	107.7	10.20	3.46	90.6%	8.7%	91.6%	3.3%
It3	94.6%	118.8	11.43	7.36	82.1%	17.8%	77.1%	22.3%
It4	80.6%	127.4	9.30	8.56	57.8%	42.2%	76.6%	22.6%
It5	57.8%	113.0	8.40	9.94	77.3%	22.3%	32.2%	67.8%
Para	83.7%	119.0	9.33	4.94	78.1%	21.8%	75.3%	23.9%
ParaSettleIt5	54.2%	111.2	10.33	10.55	75.4%	24.1%	30.3%	69.3%

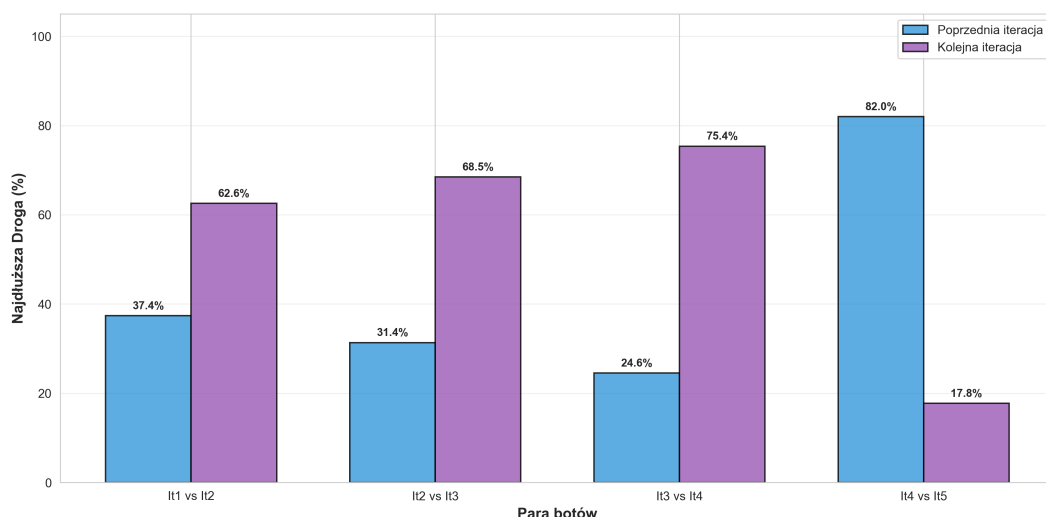
Tabela 8.4: Wyniki AlphaBetaPlayer vs przeciwnicy (karty rozwoju, produkcja)

Opponent	AvgDevCards (AB)	AvgDevCards (Opp)	ProdScore (AB)	ProdScore (Opp)
It1	2.4	0.6	1175.6	262.3
It2	2.5	0.5	1180.0	274.4
It3	2.5	1.8	1163.8	607.4
It4	2.6	1.9	1110.6	869.9
It5	2.1	2.8	1063.6	953.9
Para	2.4	2.4	1084.5	504.2
ParaSettleIt5	2.0	2.9	1032.5	996.2

Kluczowe wnioski

Monotoniczny spadek skuteczności

Wyniki pokazują wyraźny, monotoniczny spadek współczynnika zwycięstw AlphaBeta wraz ze wzrostem siły przeciwnika: od dominacji przeciwko słabym botom



Rysunek 8.7: Najdłuższa Droga (LR%) w parach iteracji.

(99.9% vs It1, 99.5% vs It2), przez silną przewagę przeciwko średnim przeciwnikom (94.6% vs It3, 80.6% vs It4), aż po niemal wyrównane starcia z najsilniejszymi botami (57.8% vs It5, 54.2% vs ParaSettleIt5). Szczególnie interesujące jest to, że AlphaBeta osiąga lepsze wyniki przeciwko ParaPlayer (83.7%) niż przeciwko It5Player (57.8%), co sugeruje, że strategia eksploracyjna ParaPlayer jest mniej efektywna przeciwko algorytmom przeszukiwania drzewa niż wyspecjalizowane heurystyki It5.

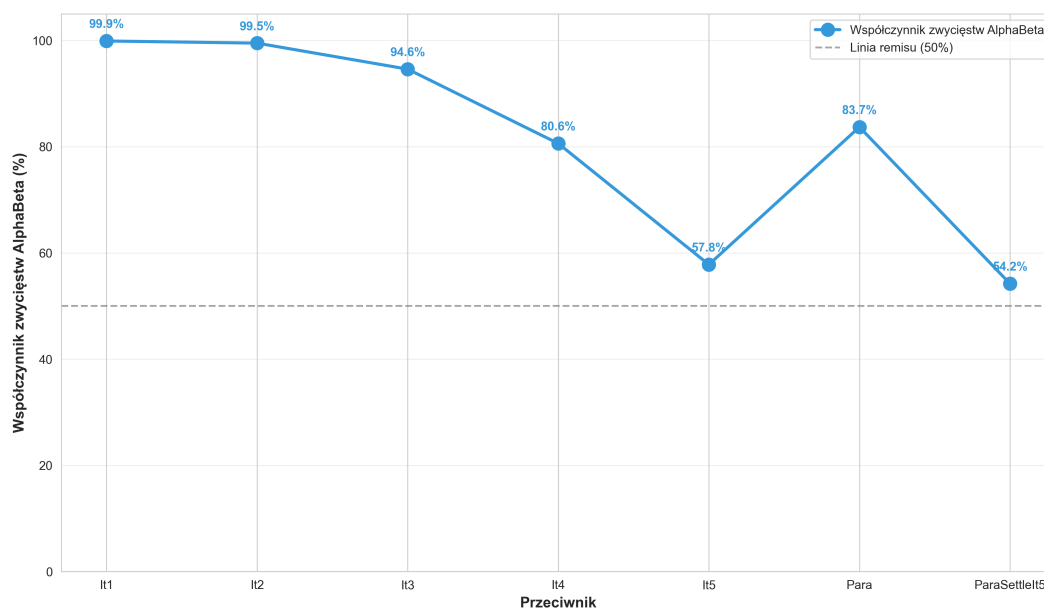
Tempo rozgrywki

Średnia liczba tur rośnie wraz ze wzrostem siły przeciwnika (od 105.0 tur vs It1 do 127.4 tur vs It4), co odzwierciedla bardziej wyrównaną rozgrywkę. Jednak przeciwko najsilniejszym przeciwnikom (It5: 113.0 tur, ParaSettleIt5: 111.2 tur) tempo nieco przyspiesza, co może wskazywać na bardziej agresywne podejście AlphaBeta w sytuacjach, gdy przewaga jest minimalna.

Dominacja w premiach strategicznych

AlphaBeta utrzymuje dominację w premii *Najdłuższa Droga* przeciwko większości przeciwników (96.3% vs It1, 90.6% vs It2, 82.1% vs It3), ale przewaga ta maleje wraz ze wzrostem siły przeciwnika. Przeciwko It4Player AlphaBeta osiąga jedynie 57.8% wykorzystania tej premii, co pokazuje, że bardziej zaawansowane boty potrafią skutecznie rywalizować również w obszarze ekspansji terytorialnej.

Szczególnie interesujący jest wzorzec dla premii *Największa Armia*: AlphaBeta dominuje przeciwko słabym i średnim przeciwnikom (92.4% vs It1, 91.6% vs It2, 77.1% vs It3), ale traci przewagę przeciwko najsilniejszym botom (32.2% vs It5, 30.3% vs ParaSettleIt5). To sugeruje, że It5Player i ParaSettleIt5 mają bardziej efektywne strategie wykorzystania kart rozwoju, które pozwalają im przejąć kontrolę nad tą premią w późniejszych fazach rozgrywki.



Rysunek 8.8: Skuteczność AlphaBetaPlayer w zależności od jakości przeciwnika

Produkcja zasobów

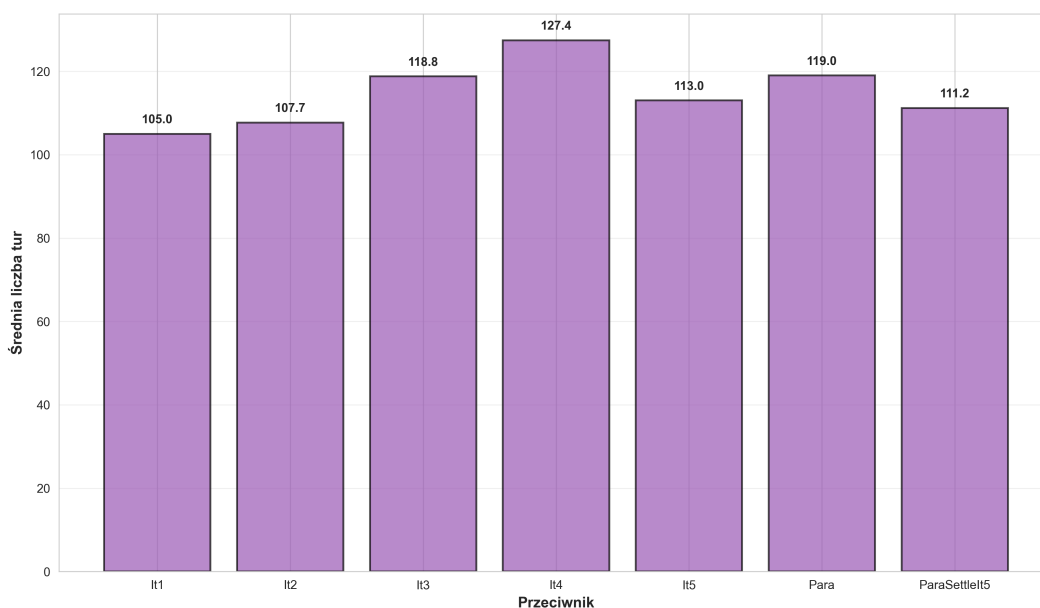
AlphaBeta utrzymuje przewagę produkcyjną przeciwko wszystkim przeciwnikom, ale różnica maleje wraz ze wzrostem siły przeciwnika. Przeciwniko It1Player różnica wynosi 913.3 punktów (1175.6 vs 262.3), podczas gdy przeciwniko ParaSettleIt5 różnica spada do zaledwie 36.3 punktów (1032.5 vs 996.2). To pokazuje, że bardziej zaawansowane boty potrafią efektywnie konkurować również w obszarze produkcji zasobów.

Dominacja mierzona przez LossVP

Metryka LossVP pokazuje interesujący wzorec: przeciwniko słabym przeciwnikom AlphaBeta przegrywa tylko w bardzo późnych fazach gry (LossVP przeciwnika: 10.00 vs It1, 10.20 vs It2), podczas gdy przeciwnicy przegrywają wcześniej (LossVP AlphaBeta: 3.38 vs It1, 3.46 vs It2). Wraz ze wzrostem siły przeciwnika różnica ta maleje, a przeciwniko najsilniejszym botom (It5, ParaSettleIt5) wartości LossVP są niemal identyczne, co potwierdza wyrównany charakter tych starć.

Granica skuteczności algorytmów przeszukiwania

Wyniki pokazują, że AlphaBetaPlayer z głębokością przeszukiwania 3 osiąga skuteczność na poziomie około 55–58% przeciwko najlepszym heurystykom (It5, ParaSettleIt5). Dalsza poprawa wymagałaby zwiększenia głębokości przeszukiwania (co wiąże się z wykładniczym wzrostem złożoności obliczeniowej) lub ulepszenia funkcji oceny pozycji. Jednocześnie wyniki pokazują, że nawet przy ograniczonej głębokości przeszukiwania algorytm jest w stanie skutecznie konkurować z najlepszymi heurystykami, co potwierdza wartość podejścia opartego na przeszukiwaniu drzewa gry.



Rysunek 8.9: Tempo rozgrywki AlphaBeta vs różni przeciwnicy

Scenariusz 4: Strategie wyspecjalizowane

Scenariusz czwarty weryfikuje skuteczność strategii wyspecjalizowanych przeciwko różnym poziomom przeciwników. W szczególności analizuje, w jakich warunkach specjalizacja w jeden aspekt rozgrywki (np. jeden surowiec, karty rozwoju, najdłuższa droga) jest efektywna oraz identyfikuje granicę skuteczności strategii skoncentrowanych na jednym celu.

OneResourcePlayer

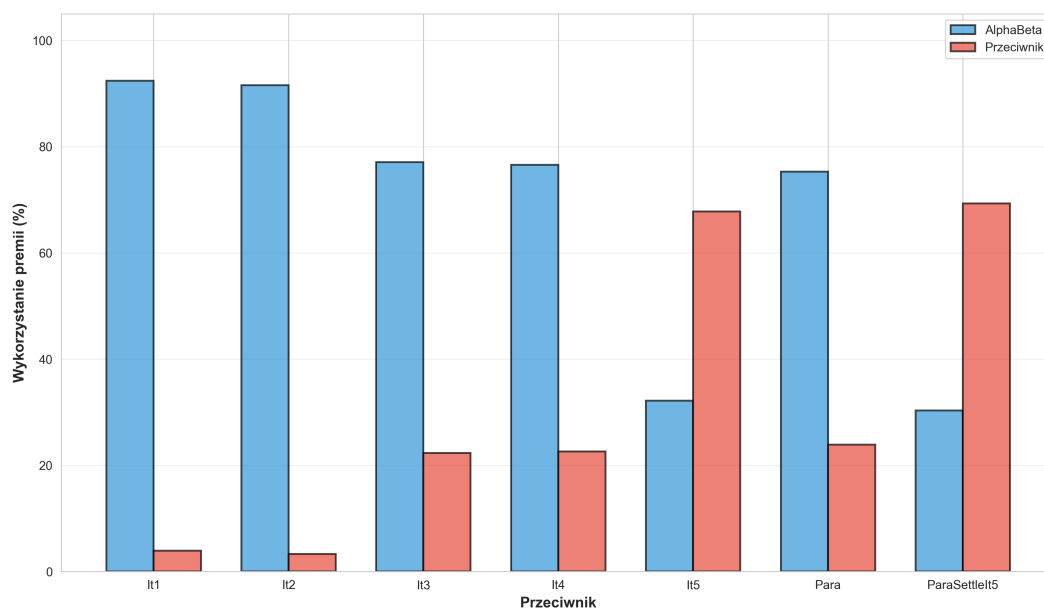
OneResourcePlayer reprezentuje strategię wyspecjalizowaną, która maksymalizuje korzyści z jednego, wybranego surowca poprzez zajęcie portu 2:1 oraz koncentrację rozwoju infrastruktury wokół tego surowca. Eksperyment weryfikuje hipotezę, czy taka specjalizacja może być skuteczna przeciwko różnym poziomom przeciwników.

Podsumowanie danych

Kluczowe wnioski

Strategia działa tylko przeciwko słabym botom

Wyniki pokazują dramatyczny spadek skuteczności strategii monosuwrowcowej wraz z poprawą jakości przeciwnika. OneResourcePlayer osiąga wysokie współczyn-



Rysunek 8.10: Częstotliwość zdobycia premii Największa Armia.

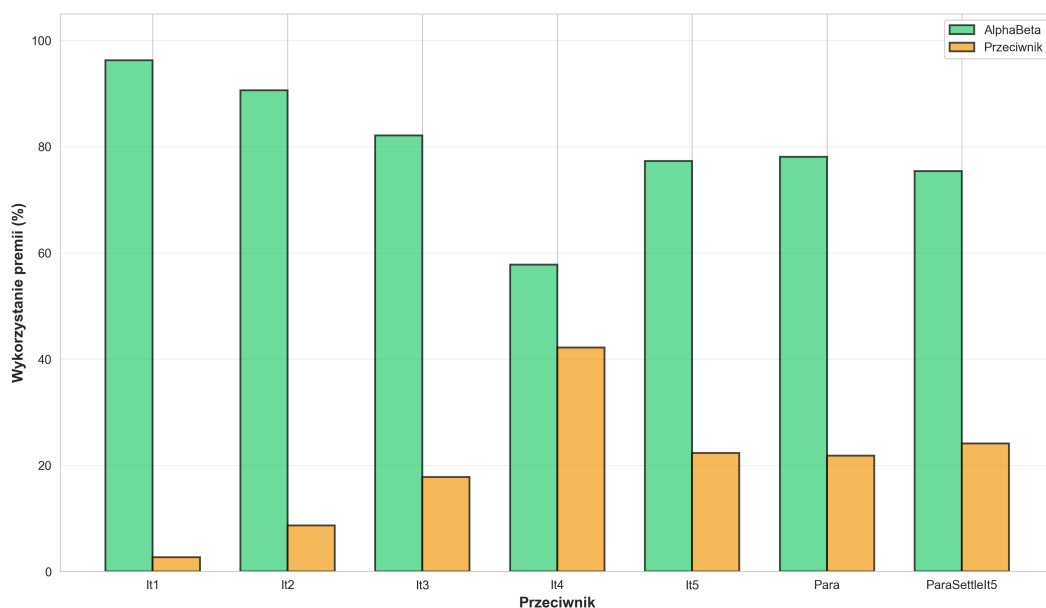
Tabela 8.5: Podsumowanie wyników OneResourcePlayer

Opponent	Win Rate	Avg Turns	LossVP (OR)	LossVP (Opp)	LR% (OR)	LR% (Opp)	LA% (OR)	LA% (Opp)
It1	98.3%	160.1	9.00	4.37	82.9%	14.7%	89.2%	8.9%
It2	95.5%	161.1	9.82	4.93	72.8%	26.1%	89.3%	8.8%
It3	56.8%	148.6	8.43	9.36	43.4%	56.5%	51.9%	47.6%
It4	36.4%	146.6	8.23	10.09	22.2%	77.8%	46.9%	51.8%
It5	19.0%	116.9	7.24	11.21	39.0%	55.8%	12.1%	87.8%
DevPlayer	51.1%	149.2	8.74	10.14	67.4%	30.1%	12.5%	87.5%
RoadPlayer	22.5%	125.9	7.45	10.28	18.3%	81.3%	22.1%	77.6%
CityRushPlayer	25.3%	117.3	7.28	10.96	42.9%	51.5%	16.4%	83.5%
Para	34.8%	127.8	7.73	6.14	32.2%	67.2%	60.6%	33.5%
ParaSettleIt5	18.2%	113.2	7.19	11.24	38.9%	54.7%	11.1%	88.7%
AlphaBeta	11.1%	111.1	6.93	10.60	14.6%	85.2%	23.2%	76.8%

niki zwycięstw przeciwko It1Player (98.3%) oraz It2Player (95.5%), jednak jego skuteczność gwałtownie spada przeciwko It3Player (56.8%), osiągając jedynie około 11–36% przeciwko zaawansowanym botom (It4, It5, ParaSettleIt5, AlphaBeta).

Istotna jest obserwacja, że It3Player stanowi punkt przełomowy. Spadek współczynnika zwycięstw z 95.5% (vs It2) do 56.8% (vs It3) pokazuje, że wprowadzenie strategii ustawień początkowych oraz aktywnego wykorzystania rozbójnika w It3 wystarcza, aby zneutralizować przewagę wynikającą ze specjalizacji w jeden surowiec. To potwierdza, że strategie wyspecjalizowane są skuteczne tylko w określonych warunkach - przeciwko słabszym przeciwnikom, którzy nie potrafią efektywnie wykorzystać mechanizmów strategicznych dostępnych w grze.

[TODO: skuteczność powraca dla para]



Rysunek 8.11: Częstotliwość zdobycia w premii Najdłuższa Droga.

DevPlayer

DevPlayer reprezentuje strategię opartą na priorytetowym skupieniu się na zakupie kart rozwoju oraz szybkim osiągnięciu związanych z nimi bonusów, takich jak premia *Największa Armia* oraz karty punktów zwycięstwa. Eksperyment weryfikuje hipotezę, czy taka strategia pośrednia może stanowić skuteczną alternatywę dla klasycznej ekspansji terytorialnej przeciwko różnym poziomom przeciwników.

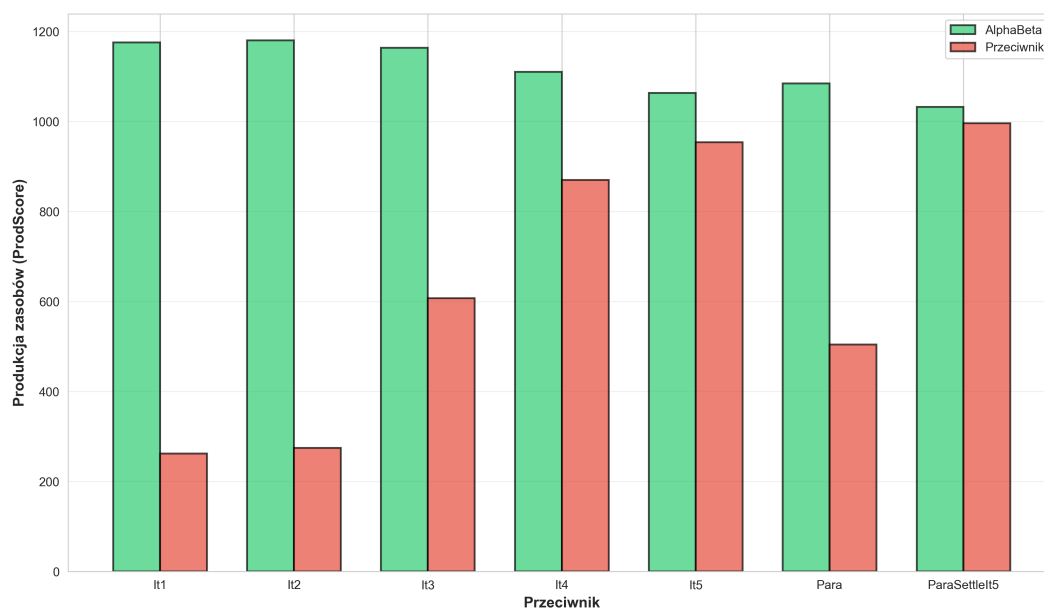
Podsumowanie danych

Tabela 8.6: Podsumowanie wyników DevPlayer

Opponent	Win Rate	Avg Turns	LossVP (Dev)	LossVP (Opp)	LR% (Dev)	LR% (Opp)	LA% (Dev)	LA% (Opp)	Avg DevC Dev	Avg DevC Opp
It1	98.8%	164.4	11.00	4.57	52.6%	36.1%	98.9%	1.0%	4.5	0.6
It2	97.5%	163.0	11.56	5.20	39.4%	53.7%	99.4%	0.6%	4.6	0.6
It3	68.1%	172.1	10.44	10.22	23.2%	76.5%	90.7%	9.3%	3.4	1.7
It4	45.8%	183.2	10.16	11.05	8.1%	91.9%	90.8%	9.2%	3.4	1.8
It5	24.9%	129.0	8.61	10.96	23.6%	72.4%	59.8%	40.2%	2.6	2.4
OneResourcePlayer	52.7%	147.5	10.38	8.46	33.3%	63.5%	89.7%	10.3%	3.5	1.6
RoadPlayer	27.1%	139.6	9.52	10.98	6.8%	92.8%	73.9%	26.1%	3.0	2.1
CityRushPlayer	31.0%	135.6	8.84	10.87	28.5%	69.1%	65.0%	35.0%	2.7	2.3
Para	58.0%	160.4	8.46	5.54	20.9%	78.6%	92.3%	7.5%	3.2	3.4
ParaSettleIt5	23.6%	129.2	8.52	11.31	23.1%	73.2%	54.4%	45.6%	2.6	2.4
AlphaBeta	18.3%	122.6	4.64	10.56	8.2%	91.6%	74.3%	25.7%	2.9	2.0

Kluczowe wnioski

Przewaga strategii w starciu z prostszymi botami



Rysunek 8.12: Porównanie produkcji zasobów AlphaBeta vs przeciwnicy.

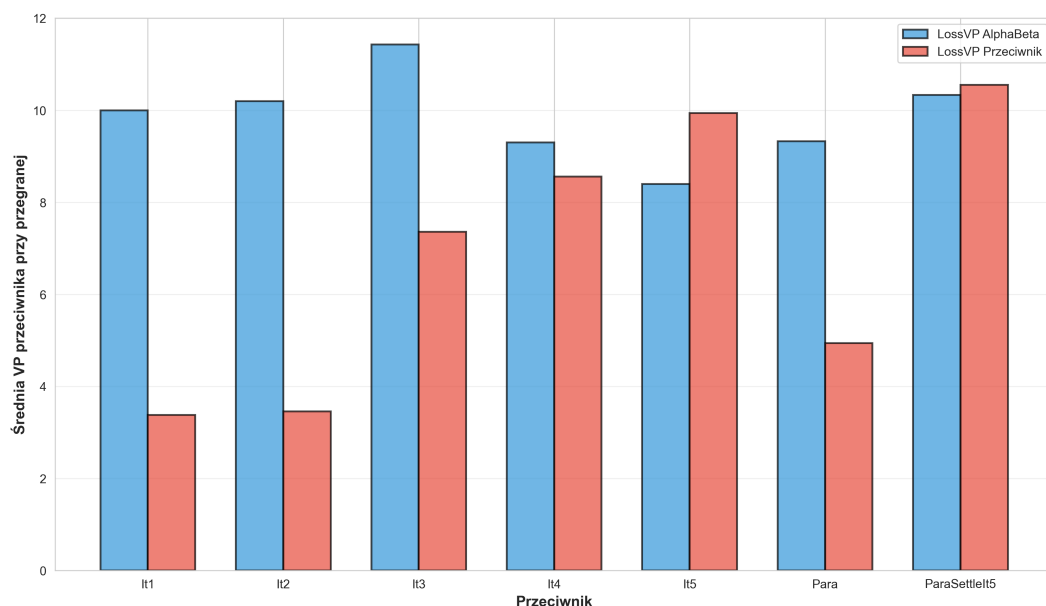
DevPlayer osiąga bardzo wysokie współczynniki zwycięstw przeciwko It1Player (98.8%) oraz It2Player (97.5%), co potwierdza skuteczność strategii opartej na kartach rozwoju w starciu z botami o niskiej jakości decyzyjnej. Kluczowym elementem sukcesu jest niemal całkowita dominacja w premii *Największa Armia* - DevPlayer zdobywa ją w 98.9% rozgrywek przeciwko It1 oraz 99.4% przeciwko It2, podczas gdy przeciwnicy osiągają tę premię w mniej niż 1% przypadków. Dodatkowo bot zakupuje średnio 4.5–4.6 kart rozwoju na rozgrywkę, co jest znacznie wyższym wynikiem niż u przeciwników (0.6 karty).

Punkt przełomowy: It3Player

Podobnie jak w przypadku OneResourcePlayer, It3Player stanowi punkt przełomowy dla skuteczności DevPlayer. Współczynnik zwycięstw spada dramatycznie z 97.5% (vs It2) do 68.1% (vs It3). Mimo że DevPlayer nadal dominuje w premii *Największa Armia* (90.7% vs 9.3%), nie przekłada się to już na tak wyraźną przewagę w rozgrywce.

Prawie remis z It4Player - granica skuteczności strategii

Najciekawszym momentem w analizie jest starcie DevPlayer vs It4Player, które kończy się niemal idealnym remisem: 45.8% vs 53.8% (z 4 rozgrywkami bez zwycięzcy). To pokazuje, że strategia oparta na kartach rozwoju osiąga granicę swojej skuteczności na poziomie It4. Warto zauważyć, że DevPlayer nadal utrzymuje wysoką dominację w premii *Największa Armia* (90.8% vs 9.2%), jednak It4Player kompensuje to poprzez zdecydowaną przewagę w premii *Najdłuższa Droga* (91.9% vs 8.1%) oraz wyższą produkcję zasobów (1201.8 vs 929.1). Średnia liczba tur wzrasta

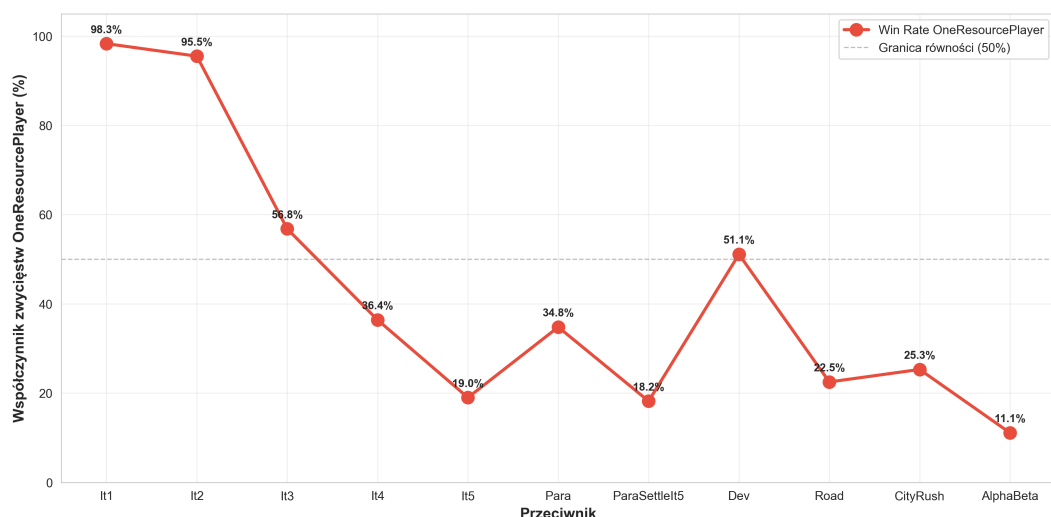


Rysunek 8.13: Średnia liczba VP gracza przy przegranej

do 183.2, co jest najwyższą wartością w całym zestawie danych.

Przedstawione wykresy pozwalają lepiej zrozumieć sposób działania strategii DevPlayer. Pierwszy wykres pokazuje, że DevPlayer konsekwentnie kupuje więcej kart rozwoju niż przeciwnicy - przeciwko It1 i It2 osiąga średnio 4.5–4.6 kart na rozgrywkę, podczas gdy przeciwnicy jedynie 0.6 karty. Jednak wraz ze wzrostem jakości przeciwnika różnica ta maleje: przeciwko It3–It4 DevPlayer kupuje 3.4 karty, a przeciwnicy już 1.7–1.8 kart. Szczególnie interesujące jest starcie z ParaPlayer, gdzie przeciwnik faktycznie kupuje więcej kart (3.4) niż DevPlayer (3.2), co pokazuje, że zaawansowane boty potrafią efektywnie włączyć strategię kartową do swojej zbalansowanej taktyki.

Drugi wykres pokazuje, że mimo częstego zdobywania przez DevPlayer premii Największa Armia przeciwko większości przeciwników, przewaga w tym aspekcie nie przekłada się bezpośrednio na proporcjonalny współczynnik zwycięstw. W starciach z It5 oraz ParaSettleIt5 odsetek zdobywania tej premii spada odpowiednio do 59.8% i 54.4%, co wskazuje, że bardziej zaawansowane boty potrafią skutecznie rywalizować również w obszarze strategii kartowej. Co więcej, przeciwko AlphaBetaPlayer dominacja DevPlayer w premii *Największa Armia* ponownie wzrasta do 74.3%, co sugeruje, że algorytm przeszukiwania drzewa gry ma inne priorytety strategiczne niż iteracyjne heurystyki.



Rysunek 8.14: Skuteczność strategii monosurowcowej w zależności od jakości przeciwnika

RoadPlayer

RoadPlayer reprezentuje strategię opartą na aktywnym rozbudowaniu sieci dróg oraz zdobyciu premii *Najdłuższa Droga* (2 punkty zwycięstwa). Eksperyment weryfikuje hipotezę, czy taka strategia może stanowić efektywną alternatywę dla klasycznego podejścia opartego na szybkim rozwoju osad i miast przeciwko różnym poziomom przeciwników.

Podsumowanie danych

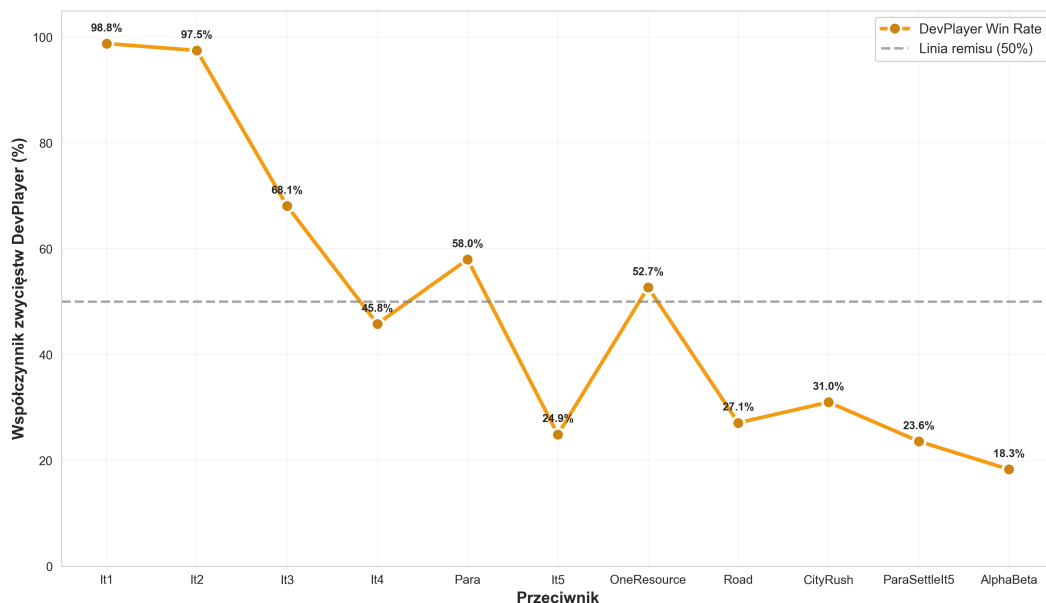
Tabela 8.7: Podsumowanie wyników RoadPlayer

Opponent	Win Rate	Avg Turns	LossVP (Road)	LossVP (Opp)	LR% (Road)	LR% (Opp)	LA% (Road)	LA% (Opp)	Avg DevC (Road)	Avg DevC (Opp)
It1	100.0%	131.2	0.00	3.58	95.9%	3.9%	96.7%	2.6%	3.5	0.7
It2	99.2%	134.2	11.57	3.96	90.8%	9.2%	96.3%	3.1%	3.6	0.6
It3	86.7%	136.1	9.68	8.23	74.3%	25.7%	79.6%	20.3%	3.0	1.8
It4	68.2%	140.2	9.54	9.42	48.3%	51.7%	75.8%	24.1%	2.9	2.0
It5	47.5%	118.9	9.06	10.40	77.1%	21.8%	33.5%	66.5%	2.2	2.8
OneResourcePlayer	80.4%	126.5	9.95	7.63	80.4%	19.1%	79.1%	20.8%	3.0	1.8
DevPlayer	72.9%	141.6	10.80	9.43	92.0%	7.8%	25.0%	75.0%	2.1	2.9
CityRushPlayer	52.9%	119.0	9.20	10.34	78.7%	20.3%	40.4%	59.6%	2.3	2.7
Para	74.2%	134.9	8.99	5.14	71.0%	28.8%	79.1%	20.2%	2.7	3.2
ParaSettleIt5	43.9%	118.3	9.01	10.84	78.3%	21.0%	28.3%	71.7%	2.1	2.9
AlphaBeta	33.7%	114.0	10.07	5.47	38.5%	61.5%	52.2%	47.8%	2.4	2.3

Kluczowe wnioski

Spadek skuteczności strategii drogowej

RoadPlayer osiąga doskonałe wyniki przeciwko prostym botom (100% vs It1, 99.2% vs It2), ale skuteczność gwałtownie spada wraz ze wzrostem poziomu przeciwnika. Strategia ta okazuje się skuteczna tylko przeciwko słabszym botom, tracąc



Rysunek 8.15: Skuteczność DevPlayer w zależności od jakości przeciwnika

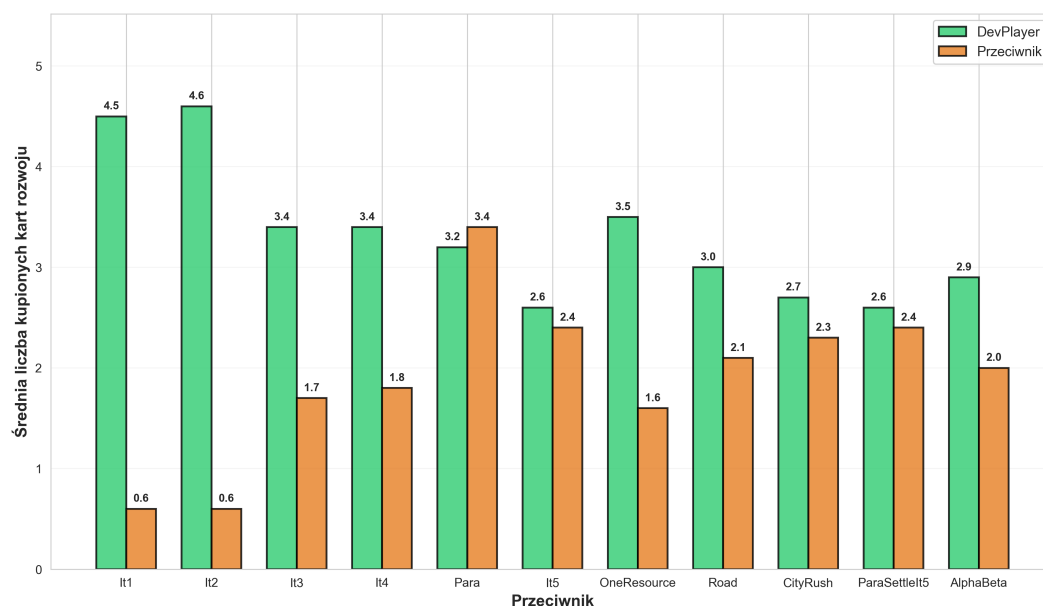
przewagę wobec zaawansowanych przeciwników (47.5% vs It5, 43.9% vs ParaSettleIt5, 33.7% vs AlphaBeta).

Punkt przełomowy: It4Player

RoadPlayer pokazuje dramatyczny spadek skuteczności między It3Player a It4Player - z 86.7% do 68.2% (spadek o 18.5 punktów procentowych). Przeciwko It5Player strategia osiąga jedynie 47.5% zwycięstw, co oznacza, że przegrywa częściej niż wygrywa. To pokazuje, że strategia drogowa, podobnie jak strategia monosuwrowcowa, jest skuteczna tylko przeciwko słabszym botom, jednak działa dłużej - Road wygrywa vs It4 (68.2%), podczas gdy OneResource wygrywa (36.4%).

Najdłuższa Droga

Szczególnie istotna jest obserwacja wyników dotyczących premii Najdłuższa Droga. RoadPlayer wygrywa 47.5% rozgrywek przeciwko It5Player, mimo że osiąga 77.1% wykorzystania Najdłuższej Drogi. Z kolei przeciwko It4Player RoadPlayer wygrywa 68.2%, mimo że osiąga 48.3% wykorzystania Najdłuższej Drogi (It4Player ma 51.7%). To pokazuje, że sama premia Najdłuższa Droga nie gwarantuje zwycięstwa - potrzebne są również osady, miasta oraz efektywne wykorzystanie innych mechanizmów gry, takich jak Największa Armia. It5Player wygrywa dzięki lepszej produkcji zasobów (ProdScore: 1044.1 vs 946.5) oraz lepszemu wykorzystaniu Największej Armii (66.5% vs 33.5%), co pokazuje, że zbalansowane strategie są bardziej skuteczne niż jednostronna specjalizacja.



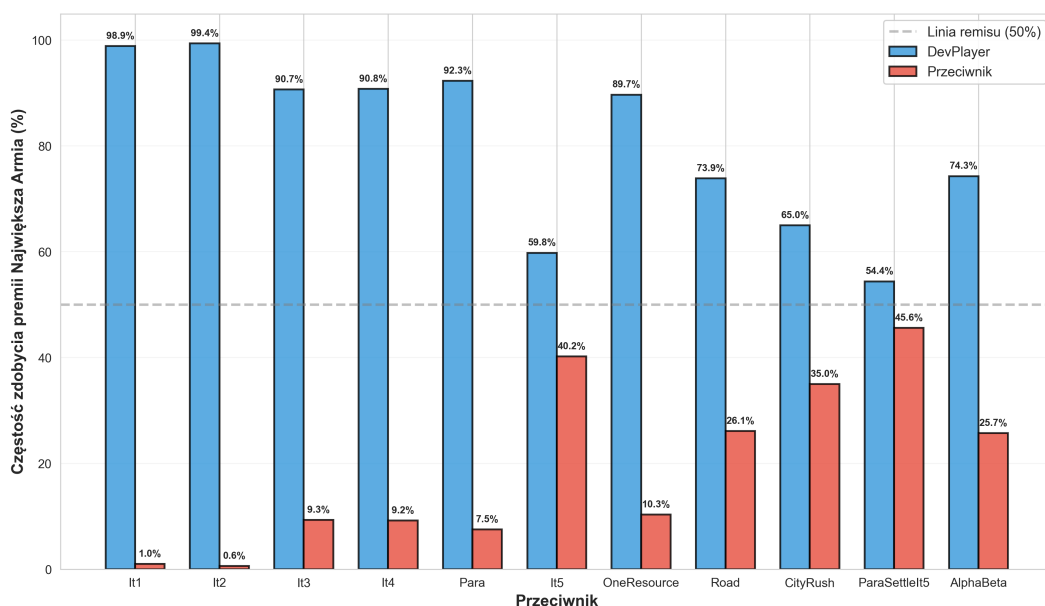
Rysunek 8.16: Średnia liczba kupionych kart rozwoju przez DevPlayer i przeciwników

Podsumowanie Scenariusza 4: Porównanie strategii wyspecjalizowanych

Analiza wyników wszystkich trzech strategii wyspecjalizowanych (OneResourcePlayer, DevPlayer, RoadPlayer) pokazuje wyraźne wzorce w ich skuteczności przeciwko różnym poziomom przeciwników. Wszystkie trzy strategie osiągają wysokie współczynniki zwycięstw przeciwko słabym botom (It1–It2: 95–100%), jednak tracą skuteczność przeciwko zaawansowanym botom (It4+: 11–68%).

RoadPlayer okazuje się najbardziej skuteczną strategią wyspecjalizowaną - osiąga 86.7% zwycięstw przeciwko It3Player oraz 68.2% przeciwko It4Player, podczas gdy OneResourcePlayer osiąga odpowiednio 56.8% i 36.4%, a DevPlayer 69.1% i 49.3%. Wyniki te wskazują, że strategia drogowa jest bardziej elastyczna niż strategia monosurowcowa czy strategia oparta na kartach rozwoju. Jest to związane z tym, że rozbudowa dróg pełni w Catanie kluczową rolę - nie tylko umożliwia zdobycie premii *Najdłuższa Droga*, ale również generuje nowe lokalizacje dla osad, które przynoszą zarówno punkty zwycięstwa, jak i dodatkową produkcję zasobów.

Drugim najlepszym graczem w tym zestawieniu jest DevPlayer. Strategia tego bota opiera się na priorytetowym wydawaniu zasobów na karty rozwoju kosztem budowy miast i osad. Należy zauważyć, że karty „Punkt Zwycięstwa” stanowią jedynie 20% talii (5 kart na 25), natomiast 52% stanowią karty Rycerza. DevPlayer, realizując swoją wąskokierunkowaną taktykę, kontynuuje zakup kart rozwoju nawet po osiągnięciu znaczącej przewagi w kartach Rycerza, podczas gdy zbalansowany bot ograniczyłby dalsze inwestycje w tym obszarze. Skutkuje to słabszym rozwojem gospodarczym. Z powyższej analizy można wywnioskować, że spośród dwóch dodat-

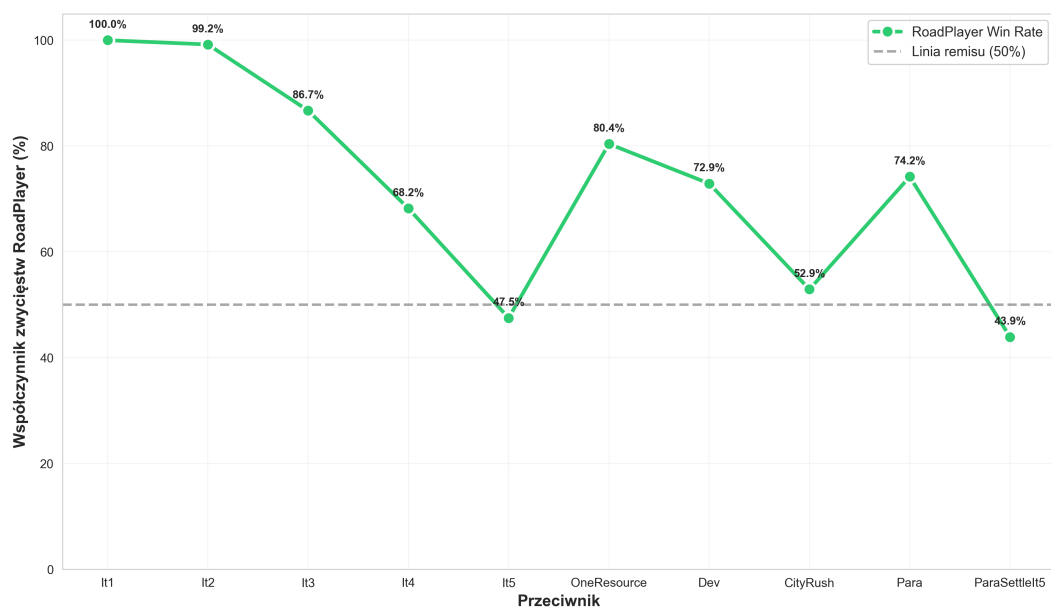


Rysunek 8.17: Częstość zdobycia premii Największa Armia przez DevPlayer i przeciwników

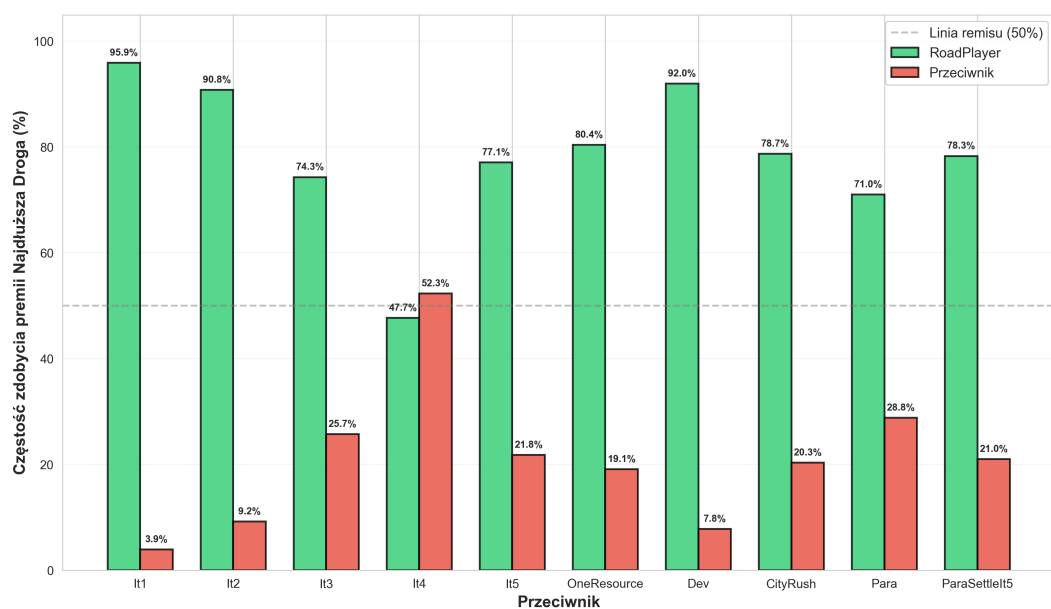
kowych premii premia *Najdłuższa Droga* okazuje się bardziej opłacalna strategicznie.

OneResourcePlayer wykazuje najslabsze wyniki spośród trzech strategii wyspecjalizowanych. Strategia monosuwrowca, mimo że zapewnia przewagę w postaci portu 2:1 oraz skoncentrowanej produkcji jednego zasobu, okazuje się zbyt jednowymiarowa - zaawansowani przeciwnicy skutecznie wykorzystują rozbójnika do blokowania kluczowych pól produkcyjnych, a przewaga w jednym surowcu nie kompensuje niedoborów w pozostałych obszarach gry. Wyniki te pokazują, że dywersyfikacja źródeł zasobów oraz elastyczność strategiczna są kluczowe w starciu z zaawansowanymi przeciwnikami.

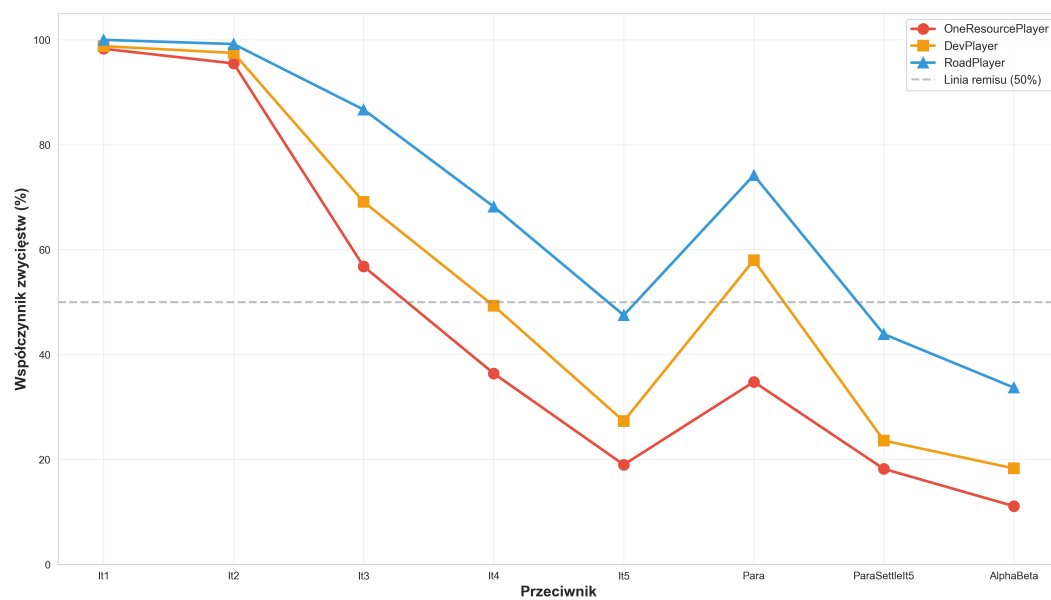
Wszystkie trzy strategie wykazują przepaść kompetencyjną między It3Player a It4Player, co potwierdza, że wprowadzenie strategii ustawień początkowych oraz aktywnego wykorzystania rozbójnika w It3 wystarcza, aby zneutralizować przewagę wynikającą z jednostronnej specjalizacji. Przeciwno zaawansowanym botom (It5, ParaSettleIt5, AlphaBeta) żadna ze strategii wyspecjalizowanych nie osiąga współczynnika zwycięstw powyżej 50%, co jednoznacznie potwierdza, że **zbalansowane strategie są bardziej skuteczne niż jednostronna specjalizacja w długoterminowej perspektywie.**



Rysunek 8.18: Skuteczność RoadPlayer w zależności od jakości przeciwnika



Rysunek 8.19: Częstość zdobycia premii Najdłuższa Droga przez RoadPlayer i przeciwników



Rysunek 8.20: Porównanie skuteczności strategii wyspecjalizowanych

Rozdział 9.

Podsumowanie i wnioski

9.1. Ocena realizacji celów pracy

W części aplikacyjnej zrealizowano cel opracowania silnika gry *Catan* w wariantcie 1 vs 1 w języku C++. Zastosowana architektura opiera się na jednoznacznym modelu stanu oraz systemie akcji wykonywanych i cofanych w sposób deterministyczny, co umożliwia szybkie symulowanie rozgrywek oraz testowanie poprawności reguł. Uzupełnieniem implementacji jest warstwa diagnostyczna (logowanie i odtwarzanie rozgrywek), która wspiera analizę zachowania botów i skraca czas wykrywania błędów.

W części badawczej przygotowano zestaw botów o zróżnicowanych strategiach (losowych, heurystycznych, wyspecjalizowanych oraz przeszukujących) i przeprowadzono eksperymenty porównawcze. Wyniki zaprezentowane w rozdziale 8 pozwalają na ocenę wpływu poszczególnych decyzji projektowych botów na skuteczność, tempo rozgrywki i charakter wygrywania.

9.2. Wnioski z części badawczej

Najistotniejszą obserwacją jest nieliniowy charakter poprawy jakości botów heurystycznych: kolejne iteracje nie zawsze przynoszą porównywalny przyrost skuteczności, a przejście $It2 \rightarrow It3$ stanowi wyraźny punkt przełomowy. Jednocześnie strategie silnie wyspecjalizowane (np. koncentracja na jednym surowcu lub jednostronne inwestowanie w karty rozwoju) mogą osiągać bardzo dobre wyniki przeciwko prostym przeciwnikom, jednak tracą skuteczność w starciu z bardziej zbalansowanymi botami.

Z punktu widzenia metodologii porównawczej istotne jest również to, że poprawa jakości botów wpływa nie tylko na win rate, ale także na średnią długość rozgrywki oraz na metryki pośrednie (np. częstość zdobywania premii Najdłuższa

Droga i Największa Armia). Wskazuje to, że porównanie strategii wymaga zestawu metryk opisujących zarówno wynik końcowy, jak i sposób prowadzenia rozgrywki.

9.3. Możliwości dalszego rozwoju systemu

W dalszym rozwoju projektu naturalnymi kierunkami są: (1) rozszerzenie reguł o pełny wariant wieloosobowy wraz z handlem pomiędzy graczami, (2) implementacja dodatkowych algorytmów decyzyjnych (np. MCTS z aproksymacją losowości i niepełnej informacji), (3) automatyzacja procesu trenowania i strojenia parametrów heurystyk oraz (4) dalsze usprawnienia wydajnościowe (profilowanie krytycznych fragmentów, optymalizacja generatorów akcji i reprezentacji stanu).

W warstwie eksperymentalnej wartościowe byłoby także uzupełnienie analiz o estymację niepewności (np. przedziały ufności dla win rate) oraz o badania w kontrolowanych warunkach (stałe konfiguracje planszy lub zbiory plansz), co pozwoliłoby lepiej rozdzielić wpływ losowości od jakości strategii.

Bibliografia

- [1] Catanatron: A High-Performance Catan Simulator, <https://docs.catanatron.com>
- [2] JSettlers - Java Settlers of Catan, <https://www.socsim.org/jsettlers/>
- [3] CatanAI GitHub Repository, <https://github.com/kvombatkere/Catan-AI>
- [4] I. Szita, G. Chaslot, P. Spronck, "Monte-Carlo Tree Search in Settlers of Catan", *Advances in Computer Games*, 2009.
- [5] S. Burre i in., "Reinforcement Learning in Settlers of Catan", *arXiv preprint arXiv:2008.07079*, 2020.
- [6] H. Cuayáhuatl i in., "Strategic Dialogue for Settlers of Catan", *arXiv preprint arXiv:1511.08099*, 2015.
- [7] Catan — Game Rules, <https://www.catan.com/understand-catan/game-rules>
- [8] Catan — Championships, <https://www.catan.com/catan-fans/championships>
- [9] Catan — Wikipedia, <https://en.wikipedia.org/wiki/Catan>
- [10] Art of Catan — Was it luck or skill?, <https://www.artofcatan.com/p/was-it-luck-or-skill>
- [11] GoogleTest — User's Guide / Reference, <https://google.github.io/googletest/reference/testing.html>
- [12] Python documentation — `tkinter`, <https://docs.python.org/3/library/tkinter.html>
- [13] Refactoring Guru — Command (design pattern), <https://refactoring.guru/design-patterns/command>
- [14] Refactoring Guru — Strategy (design pattern), <https://refactoring.guru/design-patterns/strategy>
- [15] Documentation for the JSON Lines text file format, <https://jsonlines.org/>